

Reuse your FLOPs: Scaling RL on Hard Problems by Conditioning on Very Off-Policy Prefixes

Amrith Setlur^{1,2,*}, Zijian Wang¹, Andrew Cohen¹, Paria Rashidinejad^{1,3,†}, Sang Michael Xie^{1,†}

¹Meta, ²Carnegie Mellon University, ³University of Southern California

*Work done at Meta, †Authors jointly advised this work.

Typical RL algorithms for reasoning LLMs waste enormous amounts of sampling compute on hard problems: they repeatedly sample *on-policy* traces yet rarely find a correct one, causing policy gradients to vanish. Instead of attempting to solve hard problems from scratch, we consider reusing past compute to bootstrap more efficient RL learning. Particularly, we aim to leverage *off-policy* traces from earlier inference on the base pre-trained model or previous RL runs to guide on-policy RL, enabling *self-improvement*. We introduce **PrefixRL**: run on-policy RL on *prefixed problems*, *i.e.*, problems appended with partial off-policy rollouts, in addition to *no-prefix* or original problems. This amplifies the learning signal while avoiding the instabilities of off-policy RL since it directly places the RL policy in states that reveal critical strategies. Theoretically, for certain off-policy datasets, PrefixRL is consistent with standard RL and improves worst-case sample complexity. More surprisingly, we find an empirical phenomenon driving performance gains from PrefixRL runs that we call **back-generalization**: models trained *only* on prefixed problems also substantially improve performance on no-prefix problems which we attribute to “favorable” function approximation in RL training of reasoning LLMs. On hard problems, PrefixRL outperforms our strongest baseline improving compute efficiency by $>2\times$ (even after accounting for the FLOPs spent on off-policy data collection). Broadly, our results imply that reallocating on-policy RL FLOPs to prefixes of old correct traces can be far better than spending them on repeatedly attempting the hard problem from scratch.

Date: January 23, 2026

Correspondence: Amrith Setlur, asetlur@cs.cmu.edu



1 Introduction

Reinforcement learning (RL) is now the de facto recipe for post-training large language models (LLMs) to reason more deeply, boosting math and coding performance (An et al., 2025; Liu et al., 2025b; Yu et al., 2025). Most successful recipes (Ahmadian et al., 2024; Guo et al., 2025) are *on-policy*: they sample rollouts from the current model and update parameters when at least one of these rollouts is correct. This paradigm breaks down on *hard problems*, for which the model has a very little chance of sampling a correct rollout and the $\text{pass}@k$ is extremely low (e.g., $\text{pass}@2k \approx 0$). In this regime, despite spending an egregiously large number of sampling FLOPs, the training starves from any useful learning signal and RL rewards plateau, questioning the compute efficiency of current RL methods on hard problems.

Yet in practice, we are rarely solving these problems for the first time: we also have access to an ever-growing dataset of *previously-spent compute* on the same (or similar) hard problems in the form of *off-policy* rollouts from earlier RL runs or inference done on various pre-trained models. Even when a problem is hard for the current policy, such vast historical data often contain a small number of correct trajectories. The natural question is how to reuse these rare successes to accelerate on-policy learning, e.g., by guiding the online RL policy towards a “good” high-level strategy.

A straightforward approach is to treat the correct off-policy rollouts as supervision: perform supervised finetuning (mid-training/continued pretraining) on the correct, off-policy rollouts followed by standard on-policy RL (Wang et al., 2025d). However, SFT on a tiny set of rare successes can lead to memorization (Chu et al., 2025) and entropy collapse, which in turn hurts exploration during the subsequent RL phase (Zhang et al., 2025a). Alternatively, one can incorporate the off-policy rollouts directly into RL via importance-weighted off-policy objectives; but this can yield biased or high-variance gradient estimates, destabilizing optimization (Liu et al., 2025a; Yan et al., 2025). Both of the mentioned methods use off-policy rollouts as target supervision, and since these off-policy rollouts are very low probability under the policy being trained, this leads to undesired behaviors or optimization instabilities.

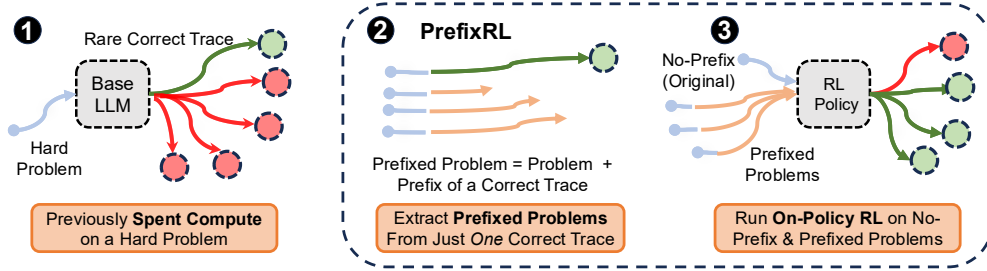


Figure 1 PrefixRL: On-Policy RL Conditioned on Off-Policy Prefixes. We leverage previously spent compute (❶) on hard problems in the form of very off-policy rare and correct rollouts obtained during inference done on the base LLM we run RL on or during past RL runs. We extract a few fixed prefixes from a single correct off-policy trace (per problem) and append each to the original hard problem to curate a dataset of prefixed problems (❷). Then, on prefixed and no-prefix (original) problems, we simply run on-policy RL (❸). PrefixRL places the RL policy in previously discovered rare states from where it a high chance of accruing positive rewards, ensuring that a "useful" learning signal never dies. Performance transfers from the prefixed to no-prefix problems via a phenomenon we refer to as back-generalization.

We propose **PrefixRL** to avoid these pitfalls: run on-policy RL after *conditioning* on prefixes of correct off-policy traces (Figure 1). *First*, given a hard problem we extract a few fixed prefixes from a single correct off-policy trace for it and curate *prefixed problems*: original problems appended with the off-policy prefixes. *Second*, on *no-prefix* (original) problems and prefixed problems we run on-policy RL. The off-policy prefixes put the current online RL policy in states that are more likely to lead to a correct answer on hard problems, reducing the variance, and increasing the learning signal’s strength from the policy gradients. However, it is not immediately clear what the effect on the bias is. Theoretically we show that when the prefixes are correct and realizable by the LLM we optimize: (i) maximizers of the PrefixRL objective also maximize performance on the no-prefix problems (standard RL objective); and (ii) since the prefixes lessen the exploration burden PrefixRL reduces suboptimality gap more quickly compared to standard RL (by a factor of token budget). Overall, PrefixRL changes the on-policy RL objective using off-policy data to only guide (gradients masked on the off-policy prefix) the RL policy’s exploration in ways that unblocks training on hard problems.

Back-generalization. Empirically, we find an interesting phenomenon behind the gains in PrefixRL. We term this *back-generalization*: training a model (with on-policy RL) on *only* prefixed-problems substantially improves test performance on the original no-prefix hard problems that were never trained on. In addition to this train/test mismatch back-generalization is distinctive for two reasons. First, we observe *generalization via shared parameters* because back-generalization alters the next-token distribution on prefixes it was never trained on (impossible in tabular RL). Second, we find that in this function approximation induced by modern-day reasoning LLMs the performance transfer via back-generalization is in some cases stronger than the typical generalization in RL (transfer from training to test environments). In an in-context learning setup, we run PrefixRL on a single problem prefixed with a different in-context problem along with its correct solution trace and find that training on a problem P1 conditioned with an in-context problem P2 improves training performance on both P1 and P2 substantially more than running RL on both P1 and P2 (see Section 4.3).

PrefixRL can discover and learn strategies beyond what is provided in the prefix. Interestingly, back-generalization does not always mean that the trained model simply learns to copy the strategy from the off-policy prefix and uses it to solve the original problem. Through controlled experiments, we find that PrefixRL is more compute efficient than standard RL at amplifying successful strategies and rejecting suboptimal strategies, even when the suboptimal strategy is in the off-policy prefix. We hypothesize that PrefixRL allows the model to more quickly try the suboptimal strategy and identify its flaws to find a better one (see Section 4.2).

PrefixRL improves both compute efficiency and final performance. In our experiments, we instantiate PrefixRL in a *self-improvement setting* by collecting a dataset of off-policy traces through large-scale rejection sampling on the base model. On hard problems in training, PrefixRL improves compute efficiency over the mid-training baseline of running SFT on the off-policy data followed by on-policy RL by $2\times$ (accounting for the compute spent on collecting the rare correct traces via rejection sampling) and training accuracy by $>45\%$ on the original hard problems without any prefixes (Figure 2 (left)). On AIME ’25, compute matched, the PrefixRL trained model improves pass@1 over 12% compared to the strongest baseline. Finally, we find that PrefixRL is not

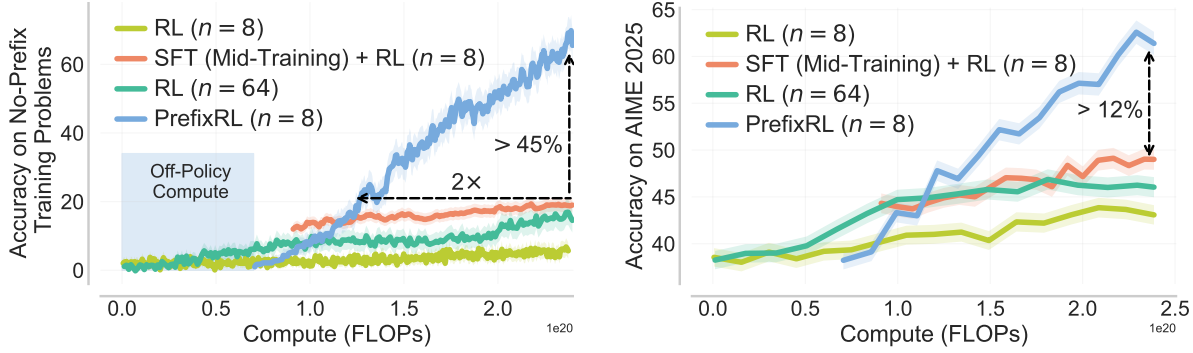


Figure 2 PrefixRL affords a self-improvement pipeline that recycles RL flops on hard problems. We instantiate PrefixRL for self-improvement by collecting a dataset of off-policy traces through large-scale rejection sampling on the base model (distilled Llama3-8B). In FLOPs-matched training, PrefixRL outperforms the strongest baseline (rejection-sampled SFT + RL): 2× higher compute efficiency (including rejection-sampling cost) and >45% higher final training accuracy on no-prefix hard problems (*left*), with reward gains transferring to standardized evals such as AIME ’25 (*right*).

restricted to a single model family: extracting off-policy prefixes from Qwen3-4B-instruct rollouts while training Llama-3.1-8B-distilled improves performance similarly to using traces from its own model family, demonstrating the flexibility of PrefixRL to the off-policy data generating source and its model size.

Broader Implications: Call for reallocation of RL FLOPs. PrefixRL reframes scaling RL on hard problems: instead of imitating rare off-policy traces, we use them to choose high-value starting states for on-policy RL, boosting the learning signal and giving a stable way to exploit very off-policy data while remaining consistent with the standard RL objective. This reallocates FLOPs away from fruitless on-policy exploration toward harvesting a modest set of prefixes and running PrefixRL, which in our experiments is both more compute-efficient and reaches higher final performance. In effect, PrefixRL turns inference logs into a first-class resource for self-improvement, continually recycling previously spent compute to push RL on ever harder problems.

2 Preliminaries

Notation. We use $\mathbf{x}$ to denote a problem and $\mathbf{y} = (y_1, \dots, y_H)$ for a response of H tokens sampled auto-regressively from an LLM π in policy class Π , where $\mathbf{y}_{:h}$ refers to a prefix of first h tokens in $\mathbf{y}$. We use $\pi^0 \in \Pi$ to denote the base (pre-trained) LLM that we want to post-train on a dataset $\mathcal{D}$ of N hard problems $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$. For each problem we have access to outcome reward function $r(\mathbf{x}_i, \mathbf{y})$ to check whether the final answer in response $\mathbf{y}$ is correct $r(\mathbf{x}_i, \mathbf{y}) = 1$ or incorrect $r(\mathbf{x}_i, \mathbf{y}) = 0$ for the problem $\mathbf{x}_i$ (e.g., by matching the boxed answer at the end of $\mathbf{y}$). We define the pass rate @ k (pass@ k) given $\mathbf{x}$ and LLM π as $\mathbb{E}_{\mathbf{y}_1, \dots, \mathbf{y}_k \sim \pi(\cdot|\mathbf{x})} \max(\{r(\mathbf{x}, \mathbf{y}_i)\}_{i=1}^k)$. We say $\mathbf{x}$ is a *hard problem* for π^0 if pass@512 under π^0 is ≈ 0 for $\mathbf{x}$. We use $J(\pi) := \sum_{\mathbf{x} \in \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi(\cdot|\mathbf{x})} r(\mathbf{x}, \mathbf{y})$ to denote the performance of π on $\mathcal{D}$. Next, we describe the iterative policy gradient RL algorithm that forms the backbone of post-training methods that optimize $J(\pi)$. For a full set of notations, please see Appendix A.

Policy gradient RL algorithms. Almost all iterative policy gradient RL algorithms for training LLMs (e.g., GRPO (Guo et al., 2025), PPO-clip (Schulman et al., 2017), REINFORCE (Ahmadian et al., 2024)) start from base LLM π^0 , and in each time-step t they perform a step of gradient ascent on $J(\pi)$, where the gradient $\nabla_{\pi} J(\pi)$ is typically computed on samples drawn from π^t , i.e., they make on-policy updates. E.g., the REINFORCE algorithm we use in our experiments updates π^t with the population gradient:

$$\nabla_{\pi} J(\pi)|_{\pi=\pi^t} = \sum_{\mathbf{x} \in \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi^t(\cdot|\mathbf{x})} [A_{\pi^t}(\mathbf{x}, \mathbf{y}) \cdot \log \pi^t(\mathbf{y}|\mathbf{x})]. \quad (2.1)$$

Following GRPO (Guo et al., 2025), the expectation in (2.1) is approximated using a Monte-carlo average over a group of n independent rollouts $\mathbf{y}_1, \dots, \mathbf{y}_n \sim \pi^t(\cdot|\mathbf{x})$ and the advantage $A_{\pi^t}(\mathbf{x}, \mathbf{y}) = r(\mathbf{x}, \mathbf{y}) - Q_{\pi^t}(\mathbf{x}, \mathbf{y})$ where $Q_{\pi^t}(\mathbf{x}, \mathbf{y}) = \mathbb{E}_{\mathbf{y}' \sim \pi^t(\cdot|\mathbf{x})} r(\mathbf{x}, \mathbf{y}')$ is also approximated with the empirical average $r(\mathbf{x}, \mathbf{y}_i) - 1/n \sum_{i \in [n]} r(\mathbf{x}, \mathbf{y}_i)$. As a result, when all n rollouts in the group for a hard problem $\mathbf{x}$ with low pass@ n fail, then the gradient update on $\mathbf{x}$ is zero and this gives rise to the *stalling regime for RL training on hard problems*.

Problem setup. Consider a base model π^0 that we post-train with RL on a dataset of verifiable problems $\mathcal{D}$. Standard on-policy RL often stalls on hard instances with low pass@ k under π^0 . Our goal is to improve RL on these prob-

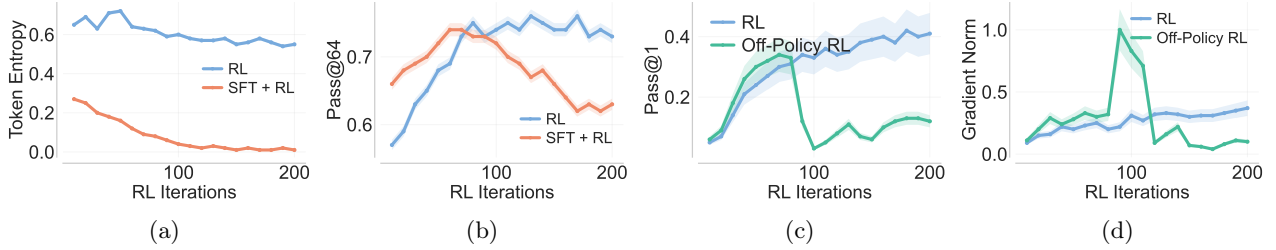


Figure 3 Supervising the policy on $\mathcal{D}_{\text{off}}$ can cause diversity collapse or training instabilities during RL: (a, b) Warm-starting the RL run by running SFT on $\mathcal{D}_{\text{off}}$ before (mid-training) reduces token entropy (a) and hurts exploration during RL (worse Pass@64 performance in (b)). (c, d) Directly using $\mathcal{D}_{\text{off}}$ during online RL by updating the current RL policy with importance-weighted off-policy traces (in addition to on-policy traces) leads to training instabilities. We see the gradient norm (clipped at 1.0) blow up during training (d) and this leads to optimization collapse (c).

lems without additional human supervision. To this end, we pose a self-improvement objective: maximize $J(\pi) := \sum_{\mathbf{x} \in \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi(\cdot|\mathbf{x})} r(\mathbf{x}, \mathbf{y})$ over $\pi \in \Pi$ with access to previously spent compute on π^0 (or on models fine-tuned from it), available in the form of inference done for various purposes like collecting mid-training data or during past RL runs. Lying in this vast dataset are rollouts done on problems in $\mathcal{D}$ and amongst them are very few rollouts that end in correct answers. As a result, we typically have a collection of rare correct (very) off-policy rollouts for problems in $\mathcal{D}$ and our main goal is to improve RL training on hard problems in $\mathcal{D}$ by leveraging previously spent compute in the form of off-policy rollouts. We situate our study in the setup of training only on hard (low pass rate) problems since that is where we expect off-policy data to be most useful though most typical RL runs train on a wider mixture.

Source of off-policy rollouts and prefix policy. To situate our study in a simplified setting, we set the source of previously spent compute to just be rejection sampling done on π^0 . Concretely, for each problem $\mathbf{x} \in \mathcal{D}$ we collect a single correct trace by continuing to sample from π^0 until we see a correct rollout. This means that, if the pass@1 under π^0 is $p_{\mathbf{x}}$ on problem $\mathbf{x}$, then in expectation we would have to sample $1/p_{\mathbf{x}}$ rollouts for $\mathbf{x}$ and doing this for every $\mathbf{x} \in \mathcal{D}$ gives us a dataset of correct off-policy rollouts we call $\mathcal{D}_{\text{off}}$, where $|\mathcal{D}_{\text{off}}| = |\mathcal{D}|$. We use μ to denote the policy that generates $\mathcal{D}_{\text{off}}$, which in this case is the rejection sampling policy over π^0 .

3 PrefixRL: On-Policy RL on Very Off-policy Prefixes

We introduce the **PrefixRL** framework that recycles old inference FLOPs in the form of very off-policy data (rare correct traces in $\mathcal{D}_{\text{off}}$) to improve RL training on hard problems. First, we discuss why typical ways of using $\mathcal{D}_{\text{off}}$ as supervision targets leads to worse performance. Then, we outline the steps in PrefixRL where, instead of training on $\mathcal{D}_{\text{off}}$, we instead *condition* the current RL policy on prefixes from $\mathcal{D}_{\text{off}}$ to guide on-policy exploration and increase the probability of success on *both* the prefixed and no-prefix (original) hard problems. Next, we theoretically prove the correctness of the PrefixRL objective and show how conditioning on off-policy prefixes can improve training rewards with fewer samples spent on exploration compared to standard RL.

Pre-RL SFT on $\mathcal{D}_{\text{off}}$ can hurt exploration during RL. A common way to improve RL on hard problems is to increase coverage of correct rollouts, for example by mid-training (SFT) on rollouts in $\mathcal{D}_{\text{off}}$ before standard RL, so the policy samples correct traces more often and obtains non-zero rewards during RL. However, while SFT boosts post-RL pass@1, pass@64 drops after about 100 iterations (Figure 3(a)). This loss of diversity is driven by a sharp entropy collapse after SFT, which decreases further during RL (Figure 3(b)), so RL mainly sharpens the distribution by cloning responses the SFT model already produces. In contrast, an early-stopped SFT checkpoint can underfit and encourage random exploration during RL (Wang et al., 2025d). One can partially mitigate this by enlarging the SFT dataset ($\mathcal{D}_{\text{off}}$) but doing so increases the upfront sampling cost.

Off-policy RL using $\mathcal{D}_{\text{off}}$ leads to training instabilities. A more direct way of improving on-policy exploration is to train on the rollouts in $\mathcal{D}_{\text{off}}$ via importance-weighted off-policy RL (Degris et al., 2012) (see Appendix D.1 for details) which accounts for the distribution shift between the current RL and sampling (rejection sampling over base LLM) policies. But this suffers from heavily biased gradients or large gradient variance (Agarwal et al., 2021) that blow up gradient norms (Section 5.2). As a result the training rewards collapse and optimization becomes unstable (Figures 3(c),(d)) as we continue to update the current RL policy on very unlikely token sequences (under it), leading to memorization of traces in $\mathcal{D}_{\text{off}}$ (Kang et al., 2024a; Setlur et al., 2024).

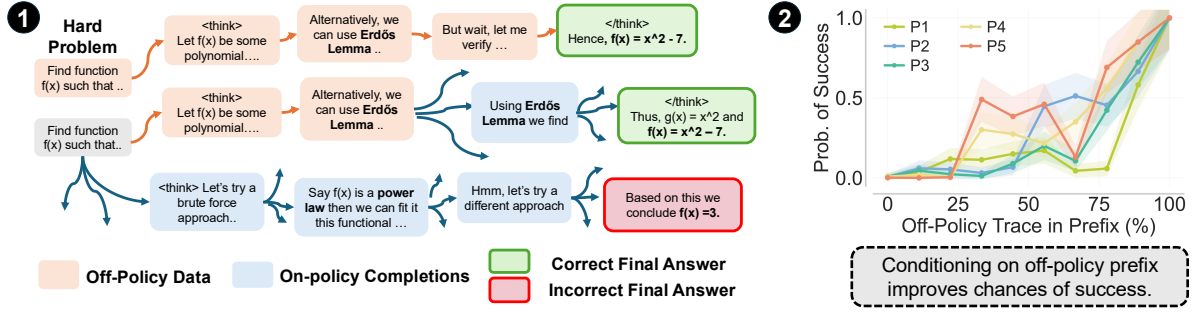


Figure 4 Prefixing on off-policy trace improves probability of future success. When we condition on prefixed-problems (hard problems appended with prefixes of very off-policy correct traces in $\mathcal{D}_{\text{off}}$) then we drastically increase the probability of future success when we condition on some key strategy revealing states (Erdős Lemma in the example in 1). For five problems (P1, ..., P5) we compute the probability of future success when their corresponding prefixed-problems (constructed using $\mathcal{D}_{\text{off}}$) contains different proportions of the off-policy trace in the prefix. In most cases we find that the probability of success stays near zero until a strategy-revealing state is visited, after which it jumps quite sharply.

PrefixRL workflow. Instead of imitating full off-policy traces, PrefixRL runs on-policy RL conditioned on a set of prefixed-problems, in addition to the no-prefix original problems. Crucially, since we condition on the off-policy data the gradients are always masked on it and we can avoid the optimization pathologies resulting policy gradients on very off-policy token. Concretely, we create a dataset of prefixed-problems $\mathcal{D}_{\text{pre}}$ by doing the following for every problem $x \in \mathcal{D}$. We take its corresponding correct trace $y^x \in \mathcal{D}_{\text{off}}$ and construct multiple prefixed problems by appending the prefix of first h tokens ($y_{1:h}^x$) in y^x to the problem x yielding the prefixed problem $\text{concat}(x, y_{1:h}^x)$. For each prefixed problem the set of prefixes are fixed throughout training and the choice of h for each corresponds to a fraction of total off-policy tokens $|y^x|$ in the correct trace. We describe how to choose this fraction in Section 5, but the key intuition here is that we should choose h such that the prefixed-problem places the base policy in a high value state, *i.e.*, a state with a high chance of success under base LLM that should only amplify during RL. Typically these are states revealing a high-level problem-solving strategy which the base LLM has little probability of sampling on its own (see Figure 4 for an illustration). In contrast, standard on-policy RL trains only on the dataset of no-prefix or original problems.

PrefixRL training objective. Given the prefixed-problems in $\mathcal{D}_{\text{pre}}$, the PrefixRL objective is similar to the reward maximization objective in standard RL except that we optimize $\pi \in \Pi$ over both prefixed-problems in $\mathcal{D}_{\text{pre}}$ and no-prefix ones in $\mathcal{D}$ within a maximum budget of H tokens. Note that the reward function $r(x, \cdot)$ for any prefixed problem $x \in \mathcal{D}_{\text{pre}}$ is identical to the reward function of the corresponding no-prefix counterpart.

$$\begin{aligned}
 \text{(PrefixRL)} \quad \max_{\pi} \quad & \underbrace{\sum_{x_{\text{pre}} \in \mathcal{D}_{\text{pre}}} \mathbb{E}_{y \sim \pi(\cdot | x_{\text{pre}})} [r(x_{\text{pre}}, y)]}_{\text{Prefixed-Problems}} + \underbrace{\sum_{x \in \mathcal{D}} \mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)]}_{\text{Standard RL: No-Prefix Problems}} \quad (3.1)
 \end{aligned}$$

Takeaway: PrefixRL conditions on off-policy traces instead of using them as supervision.

- The correct traces in $\mathcal{D}_{\text{off}}$ serve as poor supervision targets: using them in the SFT phase hurts exploration during RL and directly updating on them via off-policy RL leads to optimization collapse.
- PrefixRL never updates the RL policy on $\mathcal{D}_{\text{off}}$ and instead runs on-policy RL conditioned on prefixed-problems (original problems appended with off-policy prefixes in $\mathcal{D}_{\text{off}}$) along with the original problems.

3.1 PrefixRL Consistency and Finite-Sample Guarantees

Prefixing problems with partial correct traces places the policy in a more informative state (for instance, through revealing critical strategies), thereby making the policy substantially more likely to arrive at the right answer and receive higher rewards. Consequently, the prefixed problems provide a stronger learning signal than their no-prefix counterparts. The central theoretical question, however, is not whether learning the prefixed problem are easier, but instead whether optimizing such *augmented* prompt set provably improves the performance on the *original* RL objective $J(\pi) = \mathbb{E}_{x \sim \mathcal{D}} [\mathbb{E}_{y \sim \pi(\cdot | x)} [r(x, y)]]$, which evaluates policies only on no-prefix prompts.

In principle, training on an altered input distribution could bias optimization away from maximizing $J(\pi)$. In this section, we show that for PrefixRL this does *not* happen as long as the prefixes come from correct traces generated by a realizable policy, and we quantify the benefits of prefixing. Concretely, we prove: (i) *objective consistency*: every maximizer of the PrefixRL objective is also a maximizer of $J(\pi)$; and (ii) *sample complexity guarantees and improvement over online RL*: for a natural policy gradient variant, PrefixRL achieves a smaller suboptimality bound, which translates to a smaller number of on-policy samples required for reaching a given accuracy on $J(\pi)$. In other words, we formally show that **PrefixRL reuses your FLOPs**: it converts information already paid for in logged prefixes into sample-complexity advantages over the standard online RL.

PrefixRL objective is consistent with standard RL. PrefixRL optimizes a mixture of unprefixed and prefixed problems (Eq. (3.1)), which may raise concern in function approximation: learning to solve the prefixed instances might come at the expense of performance on the original (unprefixed) instances. We make the following assumption that the prefixes are taken from the correct traces generated by a realizable policy.

Assumption 3.1 (Realizability and correctness of off-policy traces). Assume that for any $(\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{off}}$: (i) the trace is correct: $r(\mathbf{x}, \mathbf{y}) = 1$, and (ii) the trace is realizable, i.e., there exists a policy $\mu \in \Pi$ such that $\mu(\mathbf{y} | \mathbf{x}) = 1$.

The next theorem shows that as long as prefixes are taken from the correct traces generated by a realizable policy, optimizing the PrefixRL objective preserves optimality on $J(\pi)$; see Appendix B.1 for a proof.

Theorem 3.2 (Consistency of the PrefixRL objective). Assume the realizability and correctness of off-policy traces (Assumption 3.1). Then, the maximizer of the PrefixRL objective in Equation (3.1) also maximizes standard RL objective $J(\pi)$.

PrefixRL enjoys lower sample complexity compared to standard RL. Having established that PrefixRL does not change the set of optimal policies, we now quantify the benefits of prefixing in terms of the number of on-policy samples needed to reach a near-optimal policy. We analyze PrefixRL by instantiating the policy optimization step to be natural policy gradient (NPG) (Algorithm 1). Concretely, each iteration of alternatives between (i) *policy evaluation* by fitting a Q -function in the class $\mathcal{F}$, using completions from the current policy and conditioned on the prefix states sampled from the off-policy rollouts, and (ii) *policy improvement* via a KL-regularized mirror-descent update of NPG. In practical, algorithms like PPO and GRPO serve as approximate implementation of the NPG policy improvement procedure.

The next theorem bounds the suboptimality of the policy $\bar{\pi}_T$ returned by PrefixRL Algorithm 1 in terms of the number of the number of policy updates T , the number of on-policy rollouts per iteration N , and a single distribution shift quantity between the initial base policy π_0 and the prefix generation behavior policy μ .¹

Theorem 3.3 (Suboptimality gap of PrefixRL). Under Assumption 3.1, let $\mathcal{D}_{\text{off}}$ be realized by $\mu \in \Pi$. For any $\delta \in [0, 1]$, with probability at least $1 - \delta$, policy $\bar{\pi}_T$ returned by PrefixRL-NPG (Algorithm 1) satisfies:

$$J(\pi^*) - J(\bar{\pi}_T) \leq \mathcal{O} \left(\sqrt{\frac{\text{KL}(\mu || \pi_0)}{T}} + \sqrt{\frac{1}{N} \cdot \log \left(\frac{T|\mathcal{F}|}{\delta} \right)} \right).$$

Proof of the above theorem can be found in Appendix B.2.² The bound consists of two terms:

- **Optimization term:** The first term captures the PrefixRL-NPG convergence rate. In particular, assuming access to an oracle policy evaluation step (i.e., $N \rightarrow \infty$ so that the second term vanishes), PrefixRL-NPG converges at a rate of $1/\sqrt{T}$. This rate is impacted by a constant that depends on the distribution shift between the initialization π_0 and prefix generator policy μ , and can be viewed as the KL distance the algorithm needs to traverse from π_0 to arrive at near optimality. Crucially, this divergence only depends on the π_0 and μ , and does not accumulate with intermediate policy iterations π_t . Consider the case that the off-policy rollouts are obtained through rejection sampling π_0 with at most R attempts per problem. In that case, the induced behavior policy μ is π_0 conditioned on success and satisfies $\text{KL}(\mu || \pi_0) = \mathcal{O}(\log R)$. Hence, the optimization term in Theorem 3.3 grows only logarithmically with the rejection budget.
- **Critic approximation term:** The second term captures the statistical error of policy evaluation, which is fitting $Q^{\pi_t} \in \mathcal{F}$ using N on-policy rollouts. This term is not impacted by any distribution-shift penalty

¹We do not require access to μ and only assume the off-policy rollouts are sampled from some realizable policy μ .

²It is straightforward to extend the result showing improvement competing with a suboptimal behavior policy μ .

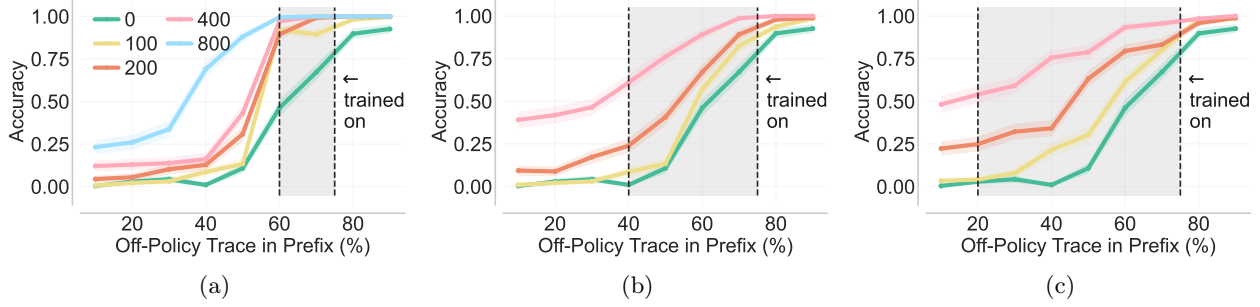


Figure 5 Back-generalization (train-test mismatch): On Llama3.1-8b-instruct we run RL only on prefixed problems whose prefix length (percent of the off-policy trace) lies in the shaded interval. At evaluation, we test across the full range of prefix lengths, including no-prefix problems (0% prefix). The performance at different RL training iterations (0, 50, 100, 400 and 800) is represented with different colors. When the mismatch is moderate, training on longer prefixes improves performance on shorter prefixes and can eventually improve no-prefix, indicating back-generalization (b, c). When training uses only very late prefixes (severe mismatch), gains concentrate near the trained region and transfer to earlier prefixes is delayed, but sufficiently long training (*e.g.*, 800 iterations) still yields improvements at no-prefix (a).

because PrefixRL collects and evaluates rollouts under the same reset distribution induced by prefixes.

In the following proposition, we analyze the worst-case performance gap between on-policy NPG and PrefixRL-NPG. The proof is deferred to Appendix B.3.

Proposition 3.4 (Worst-case separation with standard RL). *Let $\bar{\pi}_T^{\text{pre}}$ be the policy returned after T PrefixRL iterations of Algorithm 1 and $\bar{\pi}_T^{\text{std}}$ be the policy after T iterations of standard RL (states in Algorithm 1 $\sim \pi_t$). Then, $\exists$ a reward function r , base LLM π^0 s.t., $J(\bar{\pi}_T^{\text{pre}}) - J(\bar{\pi}_T^{\text{std}}) \geq 1 - (TN \cdot e^{-H})$ for $TN = o(e^H)$.*

Takeaway: PrefixRL is objective-consistent with standard RL but has better sample complexity.

- Maximizers of the PrefixRL objective also maximize $J(\pi)$ whenever all traces in $\mathcal{D}_{\text{off}}$ are correct.
- The sample complexity of the natural policy gradient (NPG) version of PrefixRL scales more favorably with long token budgets compared to the NPG version of standard RL when traces in $\mathcal{D}_{\text{off}}$ are correct.

4 Back-Generalization Boosts the Learning Signal in PrefixRL

In Section 3.1, we showed that PrefixRL is a consistent objective that leads to the same optimal solution and can also be more efficient than standard RL. We show in this section that an empirical phenomenon we call **back-generalization** is another source of performance improvement in PrefixRL that is not explained by our theory. Consider a straightforward *stitching* argument as a potential source of improvement (Zhang et al., 2025b; Qu et al., 2025a). According to the stitching argument, the model learns to complete problems more successfully by training on prefixed problems, and this improves the performance on no-prefix problems since the pass rate given any prefix that the model may generate will increase. Moreover, having good performance on prefixed problems reduces the no-prefix problem to finding the right prefix to start with. Note that this stitching argument would still hold in a tabular policy setting.

In this section, we ask whether the performance transfer from prefixed to no-prefix problems goes beyond stitching. In particular, we find that there is strong transfer from later to earlier states that is solely due to function approximation in RL. We test this **back-generalization** phenomenon in three settings: (i) by training *only* on the prefixed problems (we omit the no-prefix problems from the training prompt set) and evaluating whether this procedure improves the performance on the no-prefix problems; (ii) RL on prefixes from the off-policy trace of a single hard problem to understand how PrefixRL influences next-token distribution on untrained states, and (ii) prefixes containing a full trace from another related problem (*i.e.*, in-context learning or meta-learning setting) to study how the relationship between the prefix and completions (suffix) affects back-generalization.

In the following sections, we seek to disentangle how back-generalization arises from function approximation in strong pre-trained models, and how the relationship between the prefixed problems, no-prefix counterparts and the RL policy’s traces on the no-prefix problems modulates the extent and reliability of transfer.

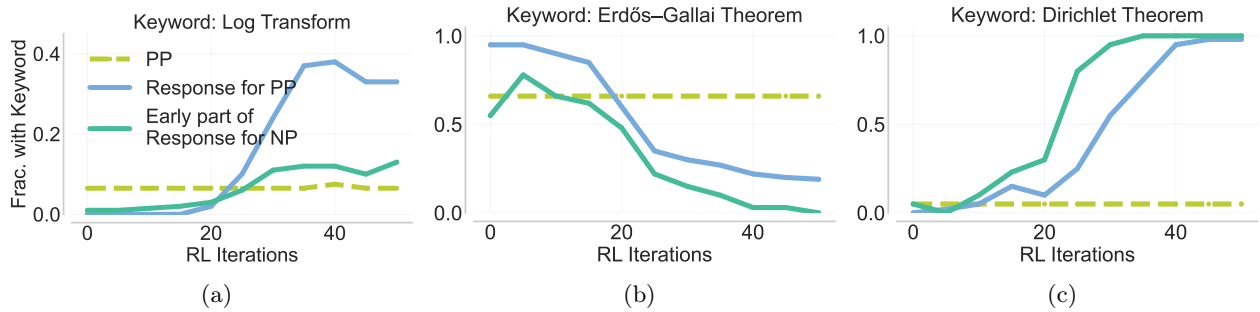


Figure 6 Strong correlation between prefixed (PP) and no-prefix (NP) responses.: For three problems, we train only on their prefixed variants and track a strategy keyword via string matching. The dashed teal curve is the keyword frequency in the prefix (fixed over training). The solid blue and green curves are the keyword frequencies in the model’s responses to PP and NP prompts, respectively. Keyword usage is tightly coupled between PP and NP responses over RL, yet not purely imitative: the model can learn new strategies or suppress prefixed ones (*e.g.*, Erdős–Gallai).

4.1 PrefixRL Improves No-Prefix Performance Even When Training Only on Prefixed-Problems

Setup. We run on-policy RL only on prefixed-problems (leaving out their no-prefix counterparts from the training set) where the length of the off-policy prefix is controlled. The prefixes are sampled such that their length is distributed uniformly between a certain fixed bandwidth of token-length percentiles, but evaluate accuracy across the full prefix spectrum, including the no-prefix endpoint (0% prefixing). We present our results in Figure 5 where the shaded region highlights the length of the prefixes trained on.

Back-generalization: generalization under a train–test distribution shift. Across multiple settings with varying prefix lengths, we see a performance improvement on no-prefix problems despite not having trained on any no-prefix problems. This transfer from prefixed to no-prefix problems is particularly notable since the prefixes are highly unlikely under the base policy. In this experiment, we can control the amount of distribution shift by the band of prefix length percentiles to train on:

- When the training mixture includes relatively short prefixes, the mismatch is moderate (Figure 5 (b,c)). In this case, performance increases first near the trained band and then progressively improves for shorter prefixes, eventually lifting no-prefix accuracy, consistent with credit propagating backward from strategy-revealing states.
- When training is restricted to very late prefixes (Figure 5(a)), the mismatch is more severe. In this case, gains initially concentrate near the trained region, with delayed improvements at earlier prefixes; nevertheless, sufficiently long training (*e.g.*, 800 iterations) still yields measurable gains at no-prefix.

Takeaway: Back-generalization drives performance improvement under a train–test mismatch.

Training only on the prefixed-problems (including only the ones with very long prefixes) improves performance on the unseen no-prefix and shorter prefix problems. We term the phenomenon behind this performance transfer under a train–test distribution shift as **back-generalization**.

4.2 PrefixRL can Discover New Strategies Beyond What is Present in the Prefixed-Problem

We show two main results: It is clear that back-generalization improves performance on unseen shorter prefixes even though we only train on longer ones, but the mechanism behind this is still unclear. To study this, we create a simplified setting where we run RL on a single problem and can qualitatively track the problem-solving strategies the model uses.

1. PrefixRL does not merely alter the distribution of the conditional completion, it changes what the model tends to generate before it, *i.e.*, it affects the LLM’s next-token distribution on earlier un-trained states, which is only possible due to function approximation. This shows that stitching does not explain back-generalization.
2. Even further, we find that the model can *discover* a strategy different from the one in the prefix and that this discovered strategy is then transferred to solving the no-prefix version that the model is not trained on.

Setup. To confirm these findings, we situate ourselves in a toy setting where we run PrefixRL on the prefixes of a single off-policy trace corresponding to a hard problem in $\mathcal{D}$. Crucially, we don’t run it on the no-prefix problem. For two hard problems, we use a keyword heuristic to extract a salient “strategy” present in the off-policy trace for each problem (see Appendix ??). Then we track the prevalence of this keyword in (i) the prefixed-problem (PP) itself, (ii) the model’s response when conditioned on that prefixed-problem (response for PP), and (iii) the early part of the model’s response on the original problem without a prefix (early part of response for no-prefix (NP)). If the presence of this keyword increases or decreases in the early part of response for NP then it means that the model’s next-token distribution has certainly changed even on the prefixes that it has not been trained on (e.g., the LLM may increase or decrease the frequency of a certain strategy in trying to solve the no-prefix problem, as reflected in the early tokens sampled from the LLM that we never update during PrefixRL).

Figure 7 probes how strategy usage evolves when training is conducted only on prefixed problems. Because the prefix is sampled from a fixed pool, the keyword frequency in the prefix remains constant by construction (dashed horizontal line). In contrast, the response curves change over training and reveal two consistent patterns:

Strategy usage is strongly correlated between prefixed and no-prefix responses as RL progresses. When the model increases (or decreases) reliance on a strategy under prefix conditioning, it tends to change in the same direction on the corresponding no-prefix problem (Figure 7 green vs. blue solid lines). This tight coupling is difficult to explain since NP problems are never trained on in this experiment, and instead suggests that PrefixRL updates internal state representations that are invoked with or without the prefix.

PrefixRL can also lead to unlearning strategies in the prefixed-problem and instead discover new ones. When comparing the strategies in the off-policy data vs. the NP (no-prefix) strategies learned by the model, we see that the model is not simply learning to copy the same strategy that is in the off-policy prefix. Instead, PrefixRL can push the model to adopt strategies that are not present in the prefix, or to actively suppress strategies that the prefix hints at. In our example, the prevalence of samples that use the Erdős–Gallai theorem illustrates this. In Figure 7(b), we see that the policy at initialization uses the Erdős–Gallai theorem close to 100% of the time on the prefixed-problems since $>50\%$ of the prefixed-problems contain references to the Erdős–Gallai theorem. Throughout training, the frequency of traces mentioning Erdős–Gallai decreases steadily on responses for the prefixed-problems, indicating that optimization can downweight the off-policy hinted strategy, and instead discover a more useful one via Dirichlet’s theorem. In Figure 7(c) we note that despite being conditioned on prefixes that contain references to the suboptimal strategy of using Erdős–Gallai, RL optimization upweights the less prevalent strategy at initialization (Dirichlet Theorem). The trends present in the responses for PP also transfer to the early part of the responses for the NP.

The above trends support the view that prefixes simply boost the training signal to accelerate training progress and exploration rather than biasing the model towards any particular solution. This supports the theoretical consistency of the PrefixRL objective in Theorem 3.2, since if PrefixRL could only learn strategies present in the off-policy prefixes, then it is unlikely to share the same optimal policy as the standard RL objective.

Takeaway: PrefixRL can discover new strategies beyond what is present in the prefixed-problems.

PrefixRL can discover strategies beyond what is present in the prefixed-problems. Benefitting from function approximation, PrefixRL alters the next-token distribution on unseen states; strategies learned and unlearned on the prefixed-problems are quickly reflected in the responses for the no-prefix ones.

4.3 Which Prefixes Back-Generalize the Most in PrefixRL?: Analysis via In-Context Learning

To study when back-generalization works effectively, we analyze it in the *in-context learning* or *meta-learning* setting, where the model is asked to solve a hard problem conditioned on a prefix that is a full trace (question followed by solution) from another hard problem. This is a clean setup that allows us to ablate the relationship between the off-policy prefix and the generated on-policy suffix through how related the in-context problems are. We find that when the in-context problem is related to the one that the model is asked to complete, performance improves on both problems, *i.e.*, including the in-context problem that the model is never trained on. In fact, this performance improvement is much stronger than the gains observed by running RL on either problems alone, indicating that performance transfer through back-generalization can be much stronger than traditional generalization in RL, where training on one environment improves performance on a related environment.

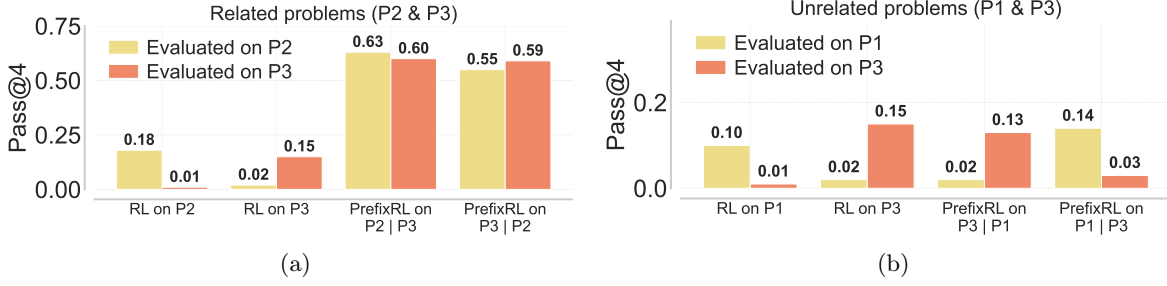


Figure 7 Performance transfer via back-generalization can be stronger than typical generalization in RL: When we prefix on the problem and full solution trace of one (in-context) problem (P2), and run Prefix RL to solve a different but related problem P3 | P2, we are able to improve performance on both P2 and P3 individually, and the performance is much higher compared to running RL either problem individually. The same holds in the opposite direction, when we run PrefixRL on P2 | P3. We do not see these gains when the in-context problem is unrelated in the case of P1 and P3.

Setup. We measure the performance of a policy trained with PrefixRL when running RL to solve a given problem with an entirely other problem (and its solution trace) in context. Consider two problem sets: (P1, P3) where P1 and P3 are unrelated sub-problems, and (P2, P3) where P2 and P3 are related and solved with the same high level strategy (see Appendix C for details on P1, P2 and P3). We choose problems that are hard for the base model, with 0% pass@4 and <3% pass@16. We also compute the performance of standard RL trained on the problems separately as a baseline.

Back-generalization occurs when the prefix and suffix are sufficiently related. When the problems are related (P2 and P3), PrefixRL on P2 given P3 achieves 63% pass@4 on P2 and 60% pass@4 on the untrained in-context problem P3. On the other hand running standard RL on P2 only, expectedly improves the performance of P2 to 0.15 but the performance transfer to P3 is fairly limited. So this suggests that the in-context problem P3 improved the learning signal on P2 when running PrefixRL on P2 | P3, enough that that back-generalization now improves performance on both P2 (without the prefix P3), and the untrained problem P3. In this case, there is little overlap in the states (in terms of the sequence of tokens) visited on the suffixes from P2 | P3 and the completions for just P3. Despite that we see a strong performance transfer which hints at the strong “positive” role played by how the reasoning model chooses to represent states visited on reasoning traces for related problems.

Back-generalization occurs weakly for an unrelated prefix and suffix. On the other hand, PrefixRL on unrelated problems (P3 | P1 or P1 | P2) is not able to outperform directly doing RL on just P3 and P1 respectively, both in terms of the test performance on the problem being trained on and the problem being fed in-context. First, the burden of exploration now falls solely over on-policy samples, weakening the performance on the trained problem. Second, the state representation for the responses generated on P3 | P1 carry little information about P1 and hence the policy learns little about “good” states from where it can find solutions for P1. This suggests that the back-generalization mechanism relies on: (i) a shared state representation between the prefix and suffix afforded by modern-day reasoning models that end up self-verifying, revising, and revisiting multiple states in the prefix when trying to complete it; (ii) learning high rewarding states in the suffixes, and (iii) back-generalizing the discovery of “good” high-rewarding states through shared representations for the prefixed and no-prefix problems.

How strong is back-generalization when prefixes are sourced from a different model family? From the above discussion it is clear that the relationship between the prefix and the traces sampled for the no-prefix counterpart have a significant bearing on the strength of back-generalization. In Appendix C (Figure 14) we present the same experiment as in Figure 5 but now with prefixed-problems derived from prefixes of correct traces from a different model family (Qwen3 prefixes for training a Llama model). When training Llama models, we find that when we prefix too long on Qwen3 prefixes performance transfer is poor compared to prefixing on traces of the same model family (Llama) — and the gap remains despite training for very long (unlike in Figure 5). On the other hand, when we prefix a wider range of lengths, the performance transfer is similar to what we find with traces from the same family. This suggests that the choice of the model family matters less if we can choose prefix distributions in ways that allow for more overlap in the states in the prefix (from Qwen) and those in the suffix (from Llama). One way of doing this is to force the Llama model to reiterate and justify some of the attempts / strategies in the prefix, and this is only feasible when we do not prefix too long.

5 Experiments and Results

In this section, we present our main empirical findings from evaluating PrefixRL on math reasoning benchmarks. First, in compute-matched comparisons on hard problems, PrefixRL consistently outperforms standard RL and SFT+RL baselines in training rewards. These improvements transfer to held-out benchmarks such as AIME, HMMT, and IMO-AnswerBench. We also ablate the source of the off-policy data, showing that PrefixRL remains effective even when the off-policy prefixes for training a Llama model are derived from correct traces produced by a different model family such as Qwen. This highlights the flexibility of PrefixRL in various practical settings. Finally, we analyze training dynamics and compare PrefixRL against other off-policy RL approaches.

Models and training problem sets. We conduct experiments on two open-weights models: **Llama-3.1-8B-Instruct** and **Qwen3-4B-Instruct**. We focus on “thinking” models, where the model outputs a chain-of-thought followed by a final answer. Note that Llama-3.1-8B-Instruct is not a thinking model, so we distill it on OpenThoughtsV3 (Guha et al., 2025) before running all our experiments. Our goal with experiments across different model families is to stress-test PrefixRL across architectures and tokenizers. However, we mainly present results with Llama while reserving the Qwen results for the Appendix.

For the training problems, we select only hard problems from DAPO (Yu et al., 2025) and OMNI-MATH (levels 6-8) (Gao et al., 2024). Here, the set of hard problems are those where 512 parallel attempts by the OpenThoughts-distilled Llama-3.1-8B model yielded incorrect answers on all attempts. This results in a fixed set of 1k training problems, which we use in all training runs.

Baselines. First, we compare against the standard (on-policy) RL baseline that does not use any off-policy data and where policy updates are done using REINFORCE (Ahmadian et al., 2024). Second, we compare against baselines that supervise the model on off-policy correct traces in $\mathcal{D}_{\text{off}}$: SFT (mid-training) on $\mathcal{D}_{\text{off}}$ followed by standard RL (SFT+RL), importance-weighted off-policy RL (Mahmood et al., 2014), and LUFFY (Yan et al., 2025), which trains with a mixed off-policy and on-policy GRPO objective. For more details on the implementations for PrefixRL and the baselines, see Appendix D.1.

Off-policy Dataset $\mathcal{D}_{\text{off}}$ and constructing prefix-problems. For each base model, we construct $\mathcal{D}_{\text{off}}$ using large-scale rejection sampling: for every training problem, we repeatedly sample rollouts until we obtain one correct trace. We explicitly account for the compute used to curate $\mathcal{D}_{\text{off}}$ in the total compute budget allocated to PrefixRL training. Given $\mathcal{D}_{\text{off}}$, we form prefixed problems as follows. For each training problem, we sample three prefixes by truncating its correct off-policy trace at a uniformly random cut point between 40% and 80% of the tokens (Figure 5). We combine the $\approx 1\text{k}$ original no-prefix training problems with these 3k prefixed problems to obtain the final prompt set for PrefixRL (see Appendix ??).

Evaluation Protocols. For all the evaluation results we report in this section, we do not condition on any off-policy prefixes and only provide the original no-prefix problem as input to the models. For all the plots, we report results with 95% confidence intervals across the problems. For details on the sampling setup for evaluation and the FLOPs accounting in compute-matched plots, see Appendix ?? and Appendix ??.

5.1 PrefixRL is More Compute-Efficient on Hard Problems Than Standard RL

For the same amount of training FLOPs, PrefixRL achieves higher rewards on the hard no-prefix training problems compared to standard RL and mid-training baselines. Crucially, this gain holds even when we account for the initial compute spent on collecting off-policy traces (for the prefixed-problems) via large-scale rejection sampling done on the base LLM. Since we are accounting for the initial parallel exploration stage (rejection sampling), this implies that PrefixRL is able to re-allocate the overall sampling and training FLOPs better than standard RL, improving training rewards on hard problems where standard RL stagnates. Importantly, we will show that these gains also transfer to held-out benchmarks.

PrefixRL efficiently achieves higher RL training rewards on hard problems than baselines. Figure 8 shows that in a compute-matched evaluation, PrefixRL achieves substantially higher no-prefix accuracy than baselines for both Llama-3.1-8B-Instruct (Figure 8 left) and Qwen-3-4B-Instruct (Figure 8 right). After accounting for the initial rejection-sampling cost, PrefixRL improves compute-efficiency by roughly $2\times$ over the strongest baseline (SFT+RL), with final no-prefix absolute accuracy gains exceeding 45% on Llama-3.1-8B and 30% on Qwen-3-4B. In contrast, standard on-policy RL and SFT+RL only slowly improve the no-prefix accuracy even when the

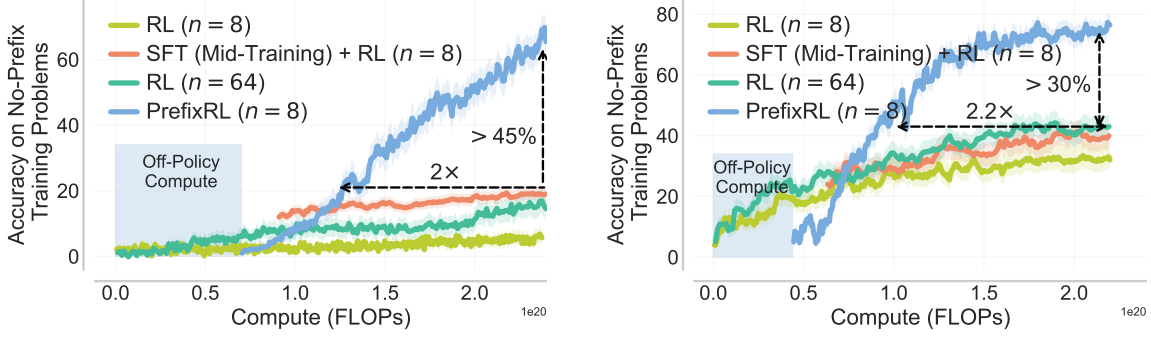


Figure 8 Training compute matched evaluation of PrefixRL. Using prefixed-problems sourced by rejection sampling the base model, we run PrefixRL on the dataset of hard training problems (low pass rate under the base model) and measure accuracy on the no-prefix training problems for **(Left)** Llama-3.1-8b-instruct and **(Right)** Qwen-3-4b-instruct. Even accounting for the initial compute spent on rejection sampling (shaded blue region), PrefixRL improves compute-efficiency by 2× over the strongest baseline with and an absolute gain of 45% with Llama and 30% with Qwen.

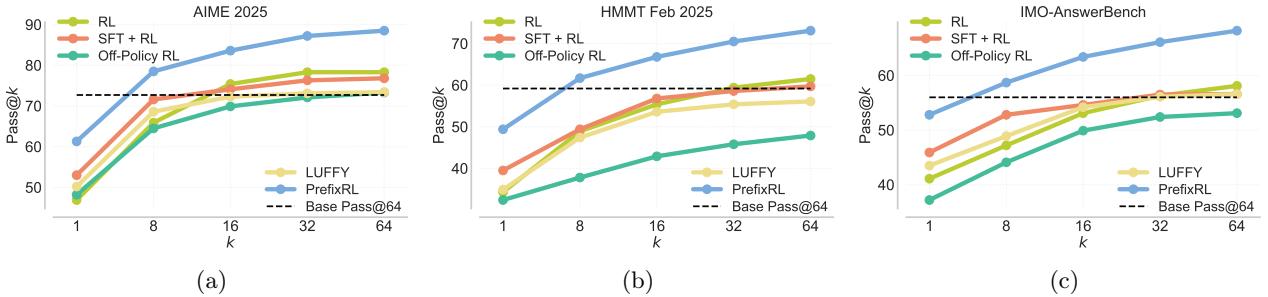


Figure 9 Held-out pass@k of PrefixRL trained Llama models: We measure pass@k (%) on AIME’25, HMMT’25 and 200 problems from IMO-AnswerBench for our distilled Llama-3.1-8B (base model) finetuned with standard on-policy RL, and PrefixRL (with and without off-policy suffix injection). Note that IMO-AnswerBench is sampled from a similar distribution to our training problems, but is still held-out. The horizontal dashed line marks the base model’s pass@64.

number of samples per input problem is increased from 8 to 64. In iteration-matched plots (Figure 10c), we see that the standard RL and SFT+RL baselines have stable training curves, so the compute gains are not explained by unstable or degenerate baseline runs.

PrefixRL improves both pass@1 and pass@k on held-out benchmarks. Figure 9 shows that across AIME’25, HMMT’25, and IMO-AnswerBench, PrefixRL consistently improves pass@k (for $k < 64$) over the baselines by over 10% absolute, including off-policy RL methods such as importance weighting (which we call Off-Policy RL) and LUFFY. The gains are visible already at $k=1$ and widen as k increases. This gain is notable since we only train on hard problems, so there is substantial transfer from the better training accuracy on hard problems to solving both easy and hard problems.

On AIME’25 with Llama-3.1-8B, PrefixRL improves pass@1 from 51.5 to 56.8, a sizable absolute gain given the benchmark’s difficulty. We observe a similar effect on HMMT’25, where pass@1 increases from 32.4 to 35.0. These small- k improvements are diagnostically important: they suggest that the model is more likely to instantiate the right high-level plan earlier in the trajectory, aligning with our “backward generalization” hypothesis (Section 4) that training on strategy-revealing states increases their prior under the unconditional start distribution. As k grows, the performance gap generally widens (e.g., on AIME’25: +6.5 points at $k=8$ and +10.1 at $k=32$). This pattern indicates that PrefixRL improves the quality of the overall search distribution: additional samples explore more promising subspaces rather than repeating low-value trajectories. In other words, PrefixRL enhances both the *mean* quality (seen in $k=1$) and the *tail* quality (seen as k increases) of rollouts.

PrefixRL increases the support of solvable problems over the course of training. How much of this gain comes from “sharpening” the model’s output distribution on problems that were already solvable with more samples (pass@k) versus an actual expansion in the support of solvable problems? Figure 10b shows that PrefixRL not

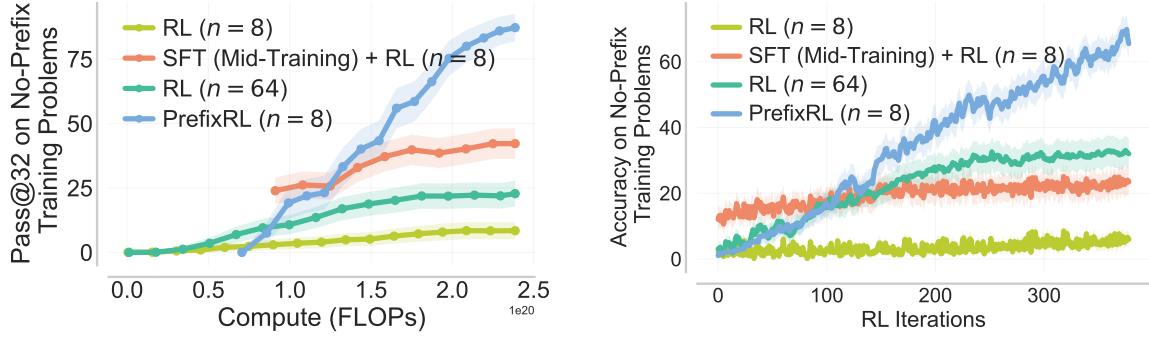


Figure 10 Pass@ k evolution on training problems and iteration-matched comparison. (Left) *New problems solved*: compute-matched pass@32 plots through the course of training indicate that PrefixRL steadily expands the set of solvable problems rather than merely converting a fixed pass@ k (for small k) into higher pass@1, whereas the baselines largely saturate on pass@32. (Right) *Fair baselines*: iteration-matched reward curves confirm stable training across methods, so the compute-matched gains are not explained by unstable or degenerate baseline runs.

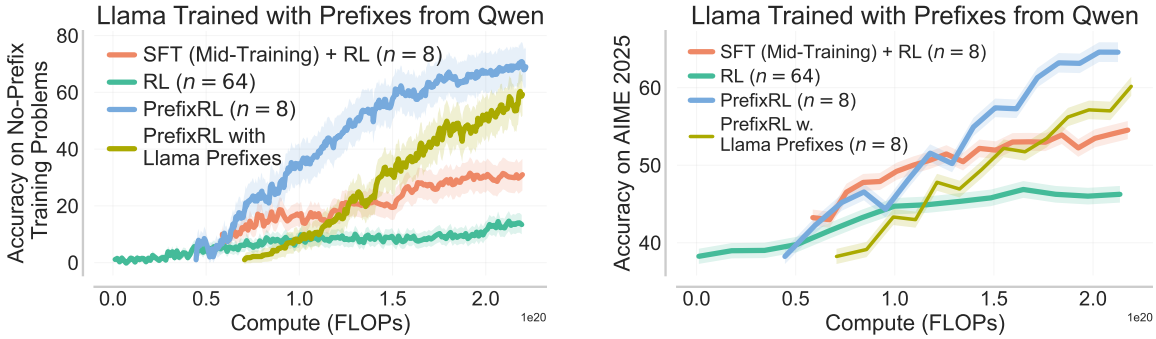


Figure 11 PrefixRL is still effective with off-policy prefixes from a different model family. We train Llama3.1-8b on prefixed-problems constructed using prefixes of rejection sampled traces from the Qwen-3-4b base LLM (left). Interestingly, even though prefixes are more out-of-distribution than those rejection sampled from Llama3.1-8b base LLM itself, they are still equally effective in improving on hard problems compared to when the prefixes are sampled from Llama3.1-8b (olive green line). We also plot the performance on AIME when we train Llama with prefixes from Qwen (right).

only improves pass@1 but also steadily improves compute-matched pass@32, while competing baselines largely saturate over the course of training. This suggests that PrefixRL increases the set of problems with non-trivial success probability rather than only converting pass@ k into higher pass@1.

PrefixRL is still effective with off-policy prefixes from a different model family. Up to now, we have used PrefixRL with prefixes from the same base LLM that we ran RL with. In many practical settings, it is easier to source off-policy prefixes from another model that is substantially different in training data or architecture; for example, from the open-source community. Motivated by this, we compare PrefixRL with mid-training and standard RL when the off-policy prefixes are sourced from a policy (Qwen3-4b-instruct) that is different from the base LLM (Llama3.1-8B-instruct) in Figure 11. We find that off-policy prefixes from Qwen, despite being more off-policy due to Qwen having a different training dataset and architecture, are similarly effective on both train and test problems as off-policy prefixes from the Llama base model for initializing on-policy RL. Note that the Llama base model required more initial compute for rejection sampling to collect the off-policy prefixes, which accounts for most of the difference in the curves. In Appendix D.2, we also find that the reverse direction is effective, where we use Llama to generate off-policy prefixes for Qwen.

5.2 Training Dynamics of PrefixRL are More Stable Compared to Typical Off-Policy Methods

By conditioning on off-policy data instead of supervising on it like importance-weighted off-policy RL, PrefixRL achieves stable RL training while leveraging off-policy data. By guiding the model with off-policy prefixes, PrefixRL has much fewer batches where all rollouts in the batch have zero reward,

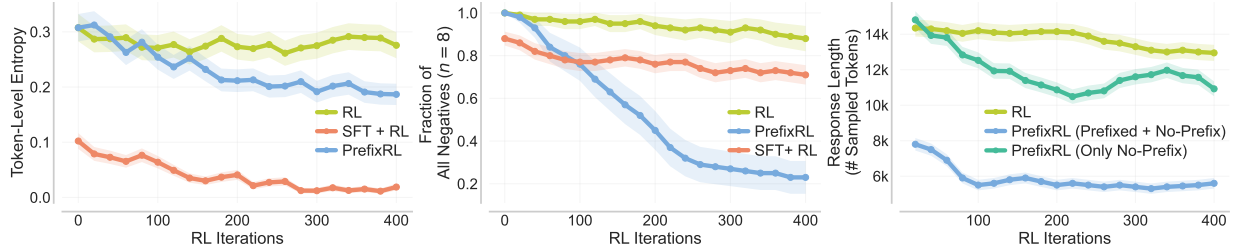


Figure 12 RL Training Dynamics. (Left) Supervised learning on off-policy data (SFT+RL) causes the model to have much lower token-level output entropy during RL than standard RL and PrefixRL, which may hurt RL exploration. (Middle) PrefixRL has much lower “all negative ratio”, or the number of prompts with all zero rewards during the training on hard problems, than standard RL. Note that the ratio is measured only on problems without any prefixes. (Right) When we look at the average response length on only no-prefix problems (green), we see PrefixRL generates shorter responses than standard RL while having higher accuracy. Blue shows the average response length across all problems, which is lower due to prefixing. This shortening is one way that PrefixRL reduces gradient variance compared to standard RL.

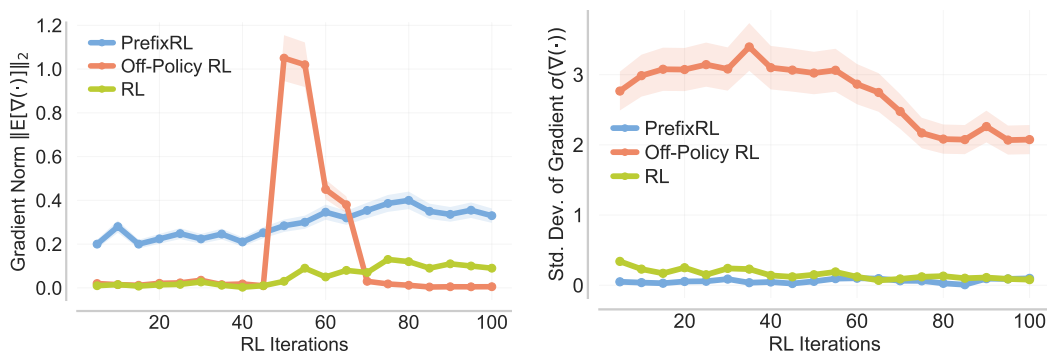


Figure 13 PrefixRL signal-to-noise ratio and safely training on off-policy suffixes. AS: Add reward plot. MX: what is RL+Off trace, we called it off-policy rl? PrefixRL simultaneously has higher gradient norm (left) and lower gradient variance (right) than both standard RL and importance-weighted off-policy RL. For off-policy RL in particular, the importance weighting causes the gradient variance to be much higher and causes a gradient norm spike, which is a sign of training instability. PrefixRL avoids these issues while still leveraging off-policy data.

PrefixRL preserves entropy for RL exploration. Figure 12 (left) shows the token-level entropy of the model output distributions during RL. Doing SFT on off-policy data causes the entropy to dramatically decrease compared the standard RL, which could hurt RL exploration. In contrast, PrefixRL preserves most of the token-level entropy while still making use of off-policy data.

Fewer all-negative batches on hard problems. Figure 12 (middle) plots the fraction of *all-negative* problems (i.e., problems in a training batch for which all k on-policy rollouts on that problem receive zero reward), measured *only* over no-prefix problems. PrefixRL consistently reduces this ratio relative to on-policy RL. This reduction reveals an underlying shift in the unconditional policy: as training on strategy-revealing states proceeds, the model becomes more likely to *enter* regions where correct completions are attainable, thereby breaking the stall regime (Section 2).

PrefixRL achieves better accuracies with less sampled tokens. Figure 12 (right) tracks the average sampled tokens per rollout across all prompts (with and without prefixes). PrefixRL maintains a lower sampled token budget per rollout while achieving higher reward rates, implying better *iteration efficiency*. Qualitatively, once the model internalizes the strategy, it reaches decisive steps earlier, which reduces unproductive wandering late in the horizon.

PrefixRL has higher signal-to-noise ratio during RL training. Figure 13 shows the gradient norm and gradient standard deviation of three methods: PrefixRL, PrefixRL with an additional importance weighted off-policy RL term on off-policy suffixes (also rejection-sampled from the base model), off-policy RL (importance weighting), and standard RL. We computed the gradient norm and standard deviation by keeping a moving average of the first and second moments of the gradient (coordinate-wise), then summing across the coordinates. For

the variance / standard deviation, this is equivalent to estimating the trace of the covariance matrix (see Appendix ??). PrefixRL simultaneously achieves a higher gradient norm and lower gradient variance than both standard RL and off-policy RL, which suffers from very noisy gradients due to importance weighting and even suffers a gradient norm spike, a sign of training instability. Recall that in Figure 12 (right), we showed that PrefixRL samples fewer tokens during training, which also contributes to the lower gradient variance. Adding an off-policy importance weighting term to PrefixRL (PrefixRL+Off-Suffix) increases the gradient variance with respect to PrefixRL, although it is able to slightly increase the gradient norm.

6 Related Work and Discussion

In our work, we present PrefixRL as a way to recycle old inference FLOPs in the form of very off-policy data for RL training of LLMs. In particular, we test the efficacy of this framework when we wish to train on hard low pass rate problems for which on-policy sampling rarely produces rollouts and exploration becomes a major bottleneck. Now, we briefly discuss a few lines of related work on off-policy RL, exploration on hard problems and a few others that improve on-policy exploration by conditioning the policy on tailored guidance and hints.

Learning from off-policy LLM rollouts. When on-policy search stalls due to over-sharpening or “over-thinking,” a common approach is to supervise on human or oracle-provided traces (Lightman et al., 2023; Corrado et al., 2024), but teacher-driven methods inherit the teacher’s capacity limit (Agarwal et al., 2024) and often require reward shaping (Yan et al., 2025), entropy control (Wang et al., 2025a), and heavy hyperparameter tuning (Zhang et al., 2025a); moreover, for hard problems, long model-compatible chains of thought are scarce and mismatches can collapse response diversity (Kang et al., 2024b). When off-policy data come from “close enough” (in KL divergence) policies as in Async RL, reuse becomes more efficient (Fu et al., 2025; Khatri et al., 2025), yet large importance weights and high gradient variance pose instability risks (Agarwal et al., 2021), so practical systems cap behavior-policy staleness to only a few RL iterations (Sheng et al., 2024). These constraints motivate approaches that do not treat off-policy trajectories as direct supervision targets; related directions condition on subgoals or plans (Hong et al., 2025), higher-level abstractions (Qu et al., 2025b), or partial solutions (Amani et al., 2025; Chen et al., 2025b; Li et al., 2025). Different from the above, PrefixRL conditions on off-policy prefixes from long-thinking traces, as opposed to training on them before or during RL. As we show in Section 3 and Section 5.2, methods that use off-policy data as supervision targets suffer from optimization pathologies. In contrast, PrefixRL benefits from them through the process of back-generalization where training on prefixed-problems with on-policy RL improve performance on the no-prefix problems.

Conditioning on hints to improve on-policy RL. Perhaps more related to the way we use off-policy data is a related line of work that augments prompts with hints or partially revealed human solutions to “guide” on-policy RL (Chen et al., 2025b; Li et al., 2025; Qu et al., 2025a). AdaBack (Amani et al., 2025) adaptively binary searches for the minimal hint that improves performance over human-written solutions and then trains on these hinted prompts, but this procedure is hard to scale to long-token budgets “thinking models” and large datasets, as claimed by the authors. Similarly QuestA (Li et al., 2025) reveals part of the answer as hints but yet again since the hinted prompts are derived from human solutions, the choice of “how much to prefix” becomes a very sensitive one as: (i) human solutions are more compressed (in number of tokens) than thinking traces from models Chen et al. (2025b); and (ii) the internal state-representations on partially revealed human solutions is not close to the states visited by the RL policy, which may preclude back-generalization Chang et al. (2024). In general, these methods are only feasible when we have access to solution traces written by a human or a more capable teacher model. In contrast, PrefixRL enables a self-improvement loop by not relying on external sources and instead reusing large scale compute (in the form of off-policy correct traces) spent on an older version of the model currently being trained with RL. Moreover, our work also disentangles the back-generalization phenomenon driving the performance transfer for PrefixRL and the above works, showing that it cannot be explained by some of the “stitching” arguments made in prior works (Zhang et al., 2025b) and that it is majorily driven by “good” function approximation afforded by strong pre-trained models.

Resetting to off-policy states in RL. The idea of “resetting” current RL policy to off-policy states is not new in RL (Kakade, 2003; Bagnell et al., 2003; Nair et al., 2018; Salimans and Chen, 2018; Yin et al., 2022; Uchendu et al., 2023; Silver et al., 2016a; Agarwal et al., 2019; Daumé III and Marcu, 2005; Daumé III et al., 2009). When resets are available, they can dramatically reshape exploration and credit assignment by repeatedly revisiting informative regions of the state space and focusing updates on long-horizon decisions; in both theory and practice,

reset can be a decisive advantage (Yin et al., 2022; Silver et al., 2016b). Our contribution is to instantiate this perspective for RL of reasoning LLMs. We show that a dataset of *correct* off-policy traces is sufficient to enable effective resets: by resetting to rejection-sampled prefixes extracted from correct long-thinking trajectories, PrefixRL makes hard, low-pass-rate problems trainable even when on-policy rollouts almost never succeed. Unlike prior reset-to-offline-data approaches that reset broadly, empirically we find that resetting to only a small set of selected prefixes can already deliver large gains; we analyze this through the lens of the back-generalization. Finally, we show that PrefixRL yields a strictly better allocation of compute, even after accounting for the up-front inference cost of collecting the off-policy traces.

Improving exploration on hard problems in LLM reasoning. Small models fine-tuned with RL can outperform much larger base models (Liu et al., 2025b; Luo et al., 2025), largely by reinforcing long chain-of-thought behaviors like self-correction (Qu et al., 2024) and reflection (Gandhi et al., 2025). Yet, without careful controls, RL often over-concentrates the policy, shrinking exploration and leaving hard instances underprobed; empirically this frequently appears as a drop in pass@ k versus the base (Yue et al., 2025; Zhao et al., 2025). One response is to regularize training to curb over-sharpening via intrinsic-motivation bonuses (Gao et al., 2025), entropy (Wang et al., 2025b), count-based signals (Song et al., 2025), or objectives that directly optimize pass@ n (Chow et al., 2024; Balashankar et al., 2025), but these still inherit sparse-reward limits and depend on easy problems for signal (He et al., 2024). A complementary thread (Setlur et al., 2025b) exploits base-model asymmetries, *e.g.*, the verification-generation gap (Setlur et al., 2025a; Song et al., 2024), and can combine with negative-gradient dynamics to chain such asymmetries across updates (Zhu et al., 2025); nevertheless, models often “under-think” (Wang et al., 2025c), persisting with wrong high-level plans despite more rollouts. In contrast the PrefixRL framework of reusing old FLOPs to source reset states avoids carefully tuned auxiliary exploration objectives by reshaping the start-state distribution while keeping the completion trained with the standard RL objective. Empirically, PrefixRL is robust to prefix choice: training on a broad range of prefixes performs comparably to an optimally chosen prefix distribution, and we do not observe the pass@ k regressions often induced by over-sharpening or over exploration with token-level entropy regularizers. In the worst case, uninformative prefixes recover standard on-policy RL.

7 Conclusion

We showed that PrefixRL can utilize off-policy prefixes as conditioning inputs to make on-policy RL more efficient on hard problems. PrefixRL is applicable to many practical settings, enabling us to recycle the FLOPs spent in previous RL or inference runs and harvest the useful high-signal states from these expensive runs. We see PrefixRL as a step beyond the typical off-policy paradigm of supervising directly on off-policy data, instead relying on the powerful back-generalization mechanism to incorporate off-policy data while doing on-policy updates.

Acknowledgements. We would like to thank Sean Bell, Ankur Pai, Aviral Kumar, Risabh Agarwal, Saurabh Garg, Wen Sun, Sergey Levine, Yuxiao Qu, Rohan Maheshwari, and Yuandong Tian for helpful discussions and thoughtful feedback on our work.

References

- Alekh Agarwal, Nan Jiang, Sham M. Kakade, and Wen Sun. Reinforcement learning: Theory and algorithms. Technical report / book draft, 2019. URL <https://rltheorybook.github.io/>. Online manuscript; frequently updated.
- Alekh Agarwal, Sham M Kakade, Jason D Lee, and Gaurav Mahajan. On the theory of policy gradient methods: Optimality, approximation, and distribution shift. *Journal of Machine Learning Research*, 22(98):1–76, 2021.
- Rishabh Agarwal, Nino Vieillard, Yongchao Zhou, Piotr Stanczyk, Sabela Ramos, Matthieu Geist, and Olivier Bachem. On-policy distillation of language models: Learning from self-generated mistakes, 2024. URL <https://arxiv.org/abs/2306.13649>.
- Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting reinforce style optimization for learning from human feedback in llms, 2024. URL <https://arxiv.org/abs/2402.14740>.
- Mohammad Hossein Amani, Aryo Lotfi, Nicolas Mario Baldwin, Samy Bengio, Mehrdad Farajtabar, Emmanuel Abbe, and Robert West. RL for reasoning by adaptively revealing rationales, 2025. URL <https://arxiv.org/abs/2506.18110>.

- Chenxin An, Zhihui Xie, Xiaonan Li, Lei Li, Jun Zhang, Shansan Gong, Ming Zhong, Jingjing Xu, Xipeng Qiu, Mingxuan Wang, and Lingpeng Kong. Polaris: A post-training recipe for scaling reinforcement learning on advanced reasoning models, 2025. URL <https://hkunlp.github.io/blog/2025/Polaris>.
- J. Andrew Bagnell, Sham Kakade, Andrew Y. Ng, and Jeff Schneider. Policy search by dynamic programming. In *Advances in Neural Information Processing Systems*, 2003. Often cited as NIPS 2003 / proceedings volume published in 2004.
- Yu Bai, Chi Jin, and Tiancheng Yu. Near-optimal reinforcement learning with self-play. *Advances in neural information processing systems*, 33:2159–2170, 2020.
- Ananth Balashankar, Ziteng Sun, Jonathan Berant, Jacob Eisenstein, Michael Collins, Adrian Hutter, Jong Lee, Chirag Nagpal, Flavien Prost, Aradhana Sinha, Ananda Theertha Suresh, and Ahmad Beirami. Infalign: Inference-aware language model alignment, 2025. URL <https://arxiv.org/abs/2412.19792>.
- Jonathan D Chang, Wenhao Shan, Owen Oertell, Kianté Brantley, Dipendra Misra, Jason D Lee, and Wen Sun. Dataset reset policy optimization for rlhf. *arXiv preprint arXiv:2404.08495*, 2024.
- Aili Chen, Aonian Li, Bangwei Gong, Binyang Jiang, Bo Fei, Bo Yang, Boji Shan, Changqing Yu, Chao Wang, Cheng Zhu, et al. Minimax-m1: Scaling test-time compute efficiently with lightning attention. *arXiv preprint arXiv:2506.13585*, 2025a.
- Justin Chih-Yao Chen, Becky Xiangyu Peng, Prafulla Kumar Choubey, Kung-Hsiang Huang, Jiaxin Zhang, Mohit Bansal, and Chien-Sheng Wu. Nudging the boundaries of llm reasoning, 2025b. URL <https://arxiv.org/abs/2509.25666>.
- Yinlam Chow, Guy Tennenholtz, Izzeddin Gur, Vincent Zhuang, Bo Dai, Sridhar Thiagarajan, Craig Boutilier, Rishabh Agarwal, Aviral Kumar, and Aleksandra Faust. Inference-aware fine-tuning for best-of-n sampling in large language models. *arXiv preprint arXiv:2412.15287*, 2024.
- Tianzhe Chu, Yuexiang Zhai, Jihan Yang, Shengbang Tong, Saining Xie, Dale Schuurmans, Quoc V Le, Sergey Levine, and Yi Ma. Sft memorizes, rl generalizes: A comparative study of foundation model post-training. *arXiv preprint arXiv:2501.17161*, 2025.
- Nicholas E. Corrado, Yuxiao Qu, John U. Balis, Adam Labiosa, and Josiah P. Hanna. Guided data augmentation for offline reinforcement learning and imitation learning, 2024. URL <https://arxiv.org/abs/2310.18247>.
- Hal Daumé III and Daniel Marcu. Learning as search optimization: Approximate large margin methods for structured prediction. In *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005.
- Hal Daumé III, John Langford, and Daniel Marcu. Search-based structured prediction. *Machine Learning*, 75:297–325, 2009. doi: 10.1007/s10994-009-5106-x.
- Thomas Degris, Martha White, and Richard S Sutton. Off-policy actor-critic. *arXiv preprint arXiv:1205.4839*, 2012.
- Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. Areal: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL <https://arxiv.org/abs/2505.24298>.
- Scott Fujimoto, David Meger, and Doina Precup. Off-policy deep reinforcement learning without exploration. *arXiv preprint arXiv:1812.02900*, 2018.
- Kanishk Gandhi, Ayush Chakravarthy, Anikait Singh, Nathan Lile, and Noah D. Goodman. Cognitive behaviors that enable self-improving reasoners, or, four habits of highly effective stars, 2025. URL <https://arxiv.org/abs/2503.01307>.
- Bofei Gao, Feifan Song, Zhe Yang, Zefan Cai, Yibo Miao, Qingxiu Dong, Lei Li, Chenghao Ma, Liang Chen, Runxin Xu, Zhengyang Tang, Benyou Wang, Daoguang Zan, Shanghaoran Quan, Ge Zhang, Lei Sha, Yichang Zhang, Xuancheng Ren, Tianyu Liu, and Baobao Chang. Omni-math: A universal olympiad level mathematic benchmark for large language models, 2024. URL <https://arxiv.org/abs/2410.07985>.
- Jingtong Gao, Ling Pan, Yejing Wang, Rui Zhong, Chi Lu, Qingpeng Cai, Peng Jiang, and Xiangyu Zhao. Navigate the unknown: Enhancing llm reasoning with intrinsic motivation guided exploration, 2025. URL <https://arxiv.org/abs/2505.17621>.
- Etash Guha, Ryan Marten, Sedrick Keh, Negin Raoof, Georgios Smyrnis, Hritik Bansal, Marianna Nezhurina, Jean Mercat, Trung Vu, Zayne Sprague, Ashima Suvana, Benjamin Feuer, Liangyu Chen, Zaid Khan, Eric Frankel, Sachin Grover, Caroline Choi, Niklas Muennighoff, Shiye Su, Wanjia Zhao, John Yang, Shreyas Pimpalgaonkar, Kartik Sharma, Charlie Cheng-Jie Ji, Yichuan Deng, Sarah Pratt, Vivek Ramanujan, Jon Saad-Falcon, Jeffrey Li, Achal Dave, Alon Albalak, Kushal Arora, Blake Wulfe, Chinmay Hegde, Greg Durrett, Sewoong Oh, Mohit Bansal, Saadia Gabriel, Aditya Grover, Kai-Wei Chang, Vaishaal Shankar, Aaron Gokaslan, Mike A. Merrill, Tatsunori Hashimoto,

- Yejin Choi, Jenia Jitsev, Reinhard Heckel, Maheswaran Sathiamoorthy, Alexandros G. Dimakis, and Ludwig Schmidt. Openthoughts: Data recipes for reasoning models, 2025. URL <https://arxiv.org/abs/2506.04178>.
- Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. Deepseek-r1 incentivizes reasoning in llms through reinforcement learning. *Nature*, 645(8081):633–638, 2025.
- Chaoqun He, Renjie Luo, Yuzhuo Bai, Shengding Hu, Zhen Leng Thai, Junhao Shen, Jinyi Hu, Xu Han, Yujie Huang, Yuxiang Zhang, Jie Liu, Lei Qi, Zhiyuan Liu, and Maosong Sun. Olympiadbench: A challenging benchmark for promoting agi with olympiad-level bilingual multimodal scientific problems, 2024. URL <https://arxiv.org/abs/2402.14008>.
- Josef Hofbauer and Sylvain Sorin. Best response dynamics for continuous zero-sum games. *Discrete and Continuous Dynamical Systems Series B*, 6(1):215, 2006.
- Joey Hong, Anca Dragan, and Sergey Levine. Planning without search: Refining frontier llms with offline goal-conditioned rl. *arXiv preprint arXiv:2505.18098*, 2025.
- Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the nineteenth international conference on machine learning*, pages 267–274, 2002.
- Sham M Kakade. A natural policy gradient. *Advances in neural information processing systems*, 14, 2001.
- Sham Machandranath Kakade. *On the Sample Complexity of Reinforcement Learning*. PhD thesis, University College London, 2003.
- Katie Kang, Amrith Setlur, Dibya Ghosh, Jacob Steinhardt, Claire Tomlin, Sergey Levine, and Aviral Kumar. What do learning dynamics reveal about generalization in llm reasoning? *arXiv preprint arXiv:2411.07681*, 2024a.
- Katie Kang, Eric Wallace, Claire Tomlin, Aviral Kumar, and Sergey Levine. Unfamiliar finetuning examples control how language models hallucinate, 2024b.
- Devvrit Khatrri, Lovish Madaan, Rishabh Tiwari, Rachit Bansal, Sai Surya Duvvuri, Manzil Zaheer, Inderjit S. Dhillon, David Brandfonbrener, and Rishabh Agarwal. The art of scaling reinforcement learning compute for llms, 2025. URL <https://arxiv.org/abs/2510.13786>.
- Vijaymohan Konda and John N. Tsitsiklis. *Actor-Critic Algorithms*. PhD thesis, USA, 2002. AAI0804543.
- Jiazheng Li, Hong Lu, Kaiyue Wen, Zaiwen Yang, Jiaxuan Gao, Hongzhou Lin, Yi Wu, and Jingzhao Zhang. Questa: Expanding reasoning capacity in llms via question augmentation. *arXiv preprint arXiv:2507.13266*, 2025.
- Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step, 2023.
- Jiacai Liu, Yingru Li, Yuqian Fu, Jiawei Wang, Qian Liu, and Yu Shen. When speed kills stability: Demystifying rl collapse from the inference-training mismatch, 2025a. URL <https://yingru.notion.site/>.
- Mingjie Liu, Shizhe Diao, Ximing Lu, Jian Hu, Xin Dong, Yejin Choi, Jan Kautz, and Yi Dong. Prorl: Prolonged reinforcement learning expands reasoning boundaries in large language models, 2025b. URL <https://arxiv.org/abs/2505.24864>.
- Michael Luo, Sijun Tan, Justin Wong, Xiaoxiang Shi, William Y. Tang, Manan Roongta, Colin Cai, Jeffrey Luo, Li Erran Li, Raluca Ada Popa, and Ion Stoica. Deepscaler: Surpassing o1-preview with a 1.5b model by scaling rl, 2025. URL <https://pretty-radio-b75.notion.site/DeepScaleR-Surpassing-O1-Preview-with-a-1-5B-Model-by-Scaling-RL-19681902c1468005bed8ca303013a4e2>. Notion Blog.
- A Rupam Mahmood, Hado P Van Hasselt, and Richard S Sutton. Weighted importance sampling for off-policy learning with linear function approximation. *Advances in neural information processing systems*, 27, 2014.
- Ashvin Nair, Praveen Srinivasan, Sam Blackwell, Cagdas Alcicek, Rory Fearon, Alessandro De Maria, Vedavyas Panneershelvam, Mustafa Suleyman, Charles Beattie, Stig Petersen, Shane Legg, and Volodymyr Mnih. Overcoming exploration in reinforcement learning with demonstrations. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, 2018. doi: 10.1109/ICRA.2018.8463167. arXiv:1709.10089.
- Yuxiao Qu, Tianjun Zhang, Naman Garg, and Aviral Kumar. Recursive introspection: Teaching language model agents how to self-improve. *arXiv preprint arXiv:2407.18219*, 2024.
- Yuxiao Qu, Amrith Setlur, Virginia Smith, Ruslan Salakhutdinov, and Aviral Kumar. How to explore to scale rl training of llms on hard problems? [urlhttps://blog.ml.cmu.edu/2025/11/26/how-to-explore-to-scale-rl-training-of-llms-on-hard-problems](https://blog.ml.cmu.edu/2025/11/26/how-to-explore-to-scale-rl-training-of-llms-on-hard-problems), 2025a. CMU MLD Blog.

- Yuxiao Qu, Anikait Singh, Yoonho Lee, Amrith Setlur, Ruslan Salakhutdinov, Chelsea Finn, and Aviral Kumar. Rlad: Training llms to discover abstractions for solving reasoning problems, 2025b. URL <https://arxiv.org/abs/2510.02263>.
- Tim Salimans and Richard Chen. Learning Montezuma’s Revenge from a single demonstration. *arXiv preprint*, 2018. arXiv:1812.03381.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *CoRR*, abs/1707.06347, 2017.
- Amrith Setlur, Saurabh Garg, Xinyang Geng, Naman Garg, Virginia Smith, and Aviral Kumar. Rl on incorrect synthetic data scales the efficiency of llm math reasoning by eight-fold. *Advances in Neural Information Processing Systems*, 37:43000–43031, 2024.
- Amrith Setlur, Nived Rajaraman, Sergey Levine, and Aviral Kumar. Scaling test-time compute without verification or rl is suboptimal, 2025a. URL <https://arxiv.org/abs/2502.12118>.
- Amrith Setlur, Matthew Y. R. Yang, Charlie Snell, Jeremy Greer, Ian Wu, Virginia Smith, Max Simchowitz, and Aviral Kumar. e3: Learning to explore enables extrapolation of test-time compute for llms, 2025b. URL <https://arxiv.org/abs/2506.09026>.
- Guangming Sheng, Chi Zhang, Zilingfeng Ye, Xibin Wu, Wang Zhang, Ru Zhang, Yanghua Peng, Haibin Lin, and Chuan Wu. Hybridflow: A flexible and efficient rlhf framework. *arXiv preprint arXiv: 2409.19256*, 2024.
- David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016a. doi: 10.1038/nature16961.
- David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *nature*, 529(7587):484–489, 2016b.
- Yuda Song, Yifei Zhou, Ayush Sekhari, J Andrew Bagnell, Akshay Krishnamurthy, and Wen Sun. Hybrid rl: Using both offline and online data can make rl efficient. *arXiv preprint arXiv:2210.06718*, 2022.
- Yuda Song, Hanlin Zhang, Carson Eisenach, Sham Kakade, Dean Foster, and Udaya Ghai. Mind the gap: Examining the self-improvement capabilities of large language models. *arXiv preprint arXiv:2412.02674*, 2024.
- Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for llm reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- Ikechukwu Uchendu, Yujia Li, Shibin Yuan, Yuke Zhu, Sergey Levine, Karol Hausman, and Chelsea Finn. Jump-start reinforcement learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, volume 202 of *Proceedings of Machine Learning Research*, 2023.
- Shenzhi Wang, Le Yu, Chang Gao, Chujie Zheng, Shixuan Liu, Rui Lu, Kai Dang, Xionghui Chen, Jianxin Yang, Zhenru Zhang, et al. Beyond the 80/20 rule: High-entropy minority tokens drive effective reinforcement learning for llm reasoning. *arXiv preprint arXiv:2506.01939*, 2025a.
- Yiping Wang, Qing Yang, Zhiyuan Zeng, Liliang Ren, Liyuan Liu, Baolin Peng, Hao Cheng, Xuehai He, Kuan Wang, Jianfeng Gao, Weizhu Chen, Shuohang Wang, Simon Shaolei Du, and Yelong Shen. Reinforcement learning for reasoning in large language models with one training example, 2025b. URL <https://arxiv.org/abs/2504.20571>.
- Yue Wang, Qiuzhi Liu, Jiahao Xu, Tian Liang, Xingyu Chen, Zhiwei He, Linfeng Song, Dian Yu, Juntao Li, Zhuosheng Zhang, et al. Thoughts are all over the place: On the underthinking of o1-like llms. *arXiv preprint arXiv:2501.18585*, 2025c.
- Zengzhi Wang, Fan Zhou, Xuefeng Li, and Pengfei Liu. Octothinker: Mid-training incentivizes reinforcement learning scaling, 2025d. URL <https://arxiv.org/abs/2506.20512>.
- Jianhao Yan, Yafu Li, Zican Hu, Zhi Wang, Ganqu Cui, Xiaoye Qu, Yu Cheng, and Yue Zhang. Learning to reason under off-policy guidance, 2025. URL <https://arxiv.org/abs/2504.14945>.
- Andrew Chi-Chin Yao. Probabilistic computations: Toward a unified measure of complexity. In *Proceedings of the 18th Annual Symposium on Foundations of Computer Science*. IEEE Computer Society, 1977. doi: 10.1109/SFCS.1977.24.

- Dong Yin, Brendan Hao, Yasin Abbasi-Yadkori, Nevena Lazic, and Csaba Szepesvári. Efficient local planning with linear function approximation. In *Proceedings of The 33rd International Conference on Algorithmic Learning Theory (ALT)*, volume 167 of *Proceedings of Machine Learning Research*, 2022.
- Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Weinan Dai, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. Dapo: An open-source llm reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in llms beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.
- Wenhao Zhang, Yuexiang Xie, Yuchang Sun, Yanxi Chen, Guoyin Wang, Yaliang Li, Bolin Ding, and Jingren Zhou. On-policy rl meets off-policy experts: Harmonizing supervised fine-tuning and reinforcement learning via dynamic weighting, 2025a. URL <https://arxiv.org/abs/2508.11408>.
- Xuechen Zhang, Zijian Huang, Yingcong Li, Chenshun Ni, Jiasi Chen, and Samet Oymak. Bread: Branched rollouts from expert anchors bridge sft & rl for reasoning. *arXiv preprint arXiv:2506.17211*, 2025b.
- Rosie Zhao, Alexandru Metereze, Sham Kakade, Cengiz Pehlevan, Samy Jelassi, and Eran Malach. Echo chamber: RL post-training amplifies behaviors learned in pretraining, 2025. URL <https://arxiv.org/abs/2504.07912>.
- Xinyu Zhu, Mengzhou Xia, Zhepei Wei, Wei-Lin Chen, Danqi Chen, and Yu Meng. The surprising effectiveness of negative reinforcement in llm reasoning, 2025. URL <https://arxiv.org/abs/2506.01347>.

Appendix

A Additional Notation

Markov decision process. We use $\mathbf{x}$ to denote an input problem and $\mathbf{y} = (y_1, \dots, y_H)$ for a response of H tokens, and if $\mathbf{y} \sim \pi(\cdot | \mathbf{x})$, then $\mathbf{y}$ is sampled auto-regressively from the LLM π fed with input $\mathbf{x}$. Each token in this response $\mathbf{y}$ belongs to a set of tokens or actions $\mathcal{A}$. The state $\mathbf{s}_h$ at time step h is given by $(\mathbf{x}, y_1, y_2, \dots, y_h)$, where the initial state $\mathbf{s}_0$ is just the problem $\mathbf{x}$. The set of states across all time steps is denoted by the class $\mathcal{S}$. We use d_h^π to denote the distribution over states $\mathbf{s}_h$ at time step h by rolling out the policy π auto-regressively for h time steps. For compactness, we write the trajectory-level log-likelihood $\log \pi(\mathbf{y} | \mathbf{x}) = \sum_{h=1}^{|\mathbf{y}|} \log \pi(y_h | \mathbf{x}, \mathbf{y}_{<h})$. For each problem we have access to outcome reward function $r(\mathbf{x}_i, \mathbf{y})$ to check whether the final answer in response $\mathbf{y}$ is correct/incorrect (1/0) for the problem $\mathbf{x}_i$ (e.g., by matching the boxed answer in the end of $\mathbf{y}$ for math problems).

Dataset of hard problems and off-policy traces. We use $\mathcal{D}$ to denote a dataset of N hard problems $\mathcal{D} = \{\mathbf{x}_i\}_{i=1}^N$. We use π^0 to denote the base pre-trained LLM that we initialize the RL algorithm, π^t as the policy after t RL iterations and $\mathcal{D}_{\text{off}}$ as the dataset of off-policy traces. Finally, we define the pass rate @ k for problem $\mathbf{x}$ and LLM π as $\mathbb{E}_{\mathbf{y}_1, \dots, \mathbf{y}_k \sim \pi(\cdot | \mathbf{x})} \max(\{r(\mathbf{x}, \mathbf{y}_i)\}_{i=1}^k)$. In the main paper, we define the set of hard problems as those with pass@ $k \approx 0$ under the base LLM π^0 . See Section 5 for how we select these low pass rate hard problems for training.

B Omitted Proofs

In this section, we present proofs for our theoretical results in Section 3.1. We begin with the proof for Theorem 3.2 which implies that the PrefixRL objective is consistent with standard RL, and any solution for our PrefixRL objective, is also a maximizer of the standard RL objective which just maximizes $J(\pi)$. Following this, we show the proof for Theorem 3.3 which bounds the suboptimality gap of an algorithm using natural policy gradient (NPG) to optimize the PrefixRL objective. Note that this is slightly different from the policy gradients we use in practice, but is nevertheless insightful in informing a formal mental model for the gains behind PrefixRL. We also provide proofs for Corollary ?? for a rejection sampled $\mathcal{D}_{\text{off}}$ dataset and Proposition 3.4 that lower bounds the performance gap between PrefixRL and standard RL in the worst case. In the end we list auxiliary lemmas useful for analysis.

B.1 Proof of Theorem 3.2

Let the standard (no-prefix) RL objective be

$$J(\pi) = \sum_{\mathbf{x} \in \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})], \quad J^* = \max_{\pi \in \Pi} J(\pi). \quad (\text{B.1})$$

For each $\mathbf{x} \in \mathcal{D}$, Assumption 3.1 gives a single correct trace $\mathbf{y}^x$ such that $(\mathbf{x}, \mathbf{y}^x) \in \mathcal{D}_{\text{off}}$. Given any cut index h , define the prefixed problem

$$\mathbf{x}_{\text{pre}} = \text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h}), \quad (\text{B.2})$$

and define its reward by evaluating the full transcript:

$$r(\mathbf{x}_{\text{pre}}, \mathbf{z}) := r(\mathbf{x}, (\mathbf{y}^x)_{1:h} \circ \mathbf{z}).$$

The PrefixRL objective is

$$J_{\text{pre}}(\pi) = \sum_{\mathbf{x}_{\text{pre}} \in \mathcal{D}_{\text{pre}}} \mathbb{E}_{\mathbf{z} \sim \pi(\cdot | \mathbf{x}_{\text{pre}})} [r(\mathbf{x}_{\text{pre}}, \mathbf{z})] + \sum_{\mathbf{x} \in \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \pi(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})]. \quad (\text{B.3})$$

A uniform upper bound. Fix any $\mathbf{x} \in \mathcal{D}$ and any prefix $(\mathbf{y}^x)_{1:h}$ used to form a prefixed problem. For any policy $\pi \in \Pi$ defined on such prefixed problems, construct a policy $\tilde{\pi} \in \Pi$ for the no-prefix problem $\mathbf{x}$ that deterministically emits the prefix $(\mathbf{y}^x)_{1:h}$ and then samples the suffix from $\pi(\cdot | \text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h}))$. By the reward definition,

$$\mathbb{E}_{\mathbf{y} \sim \tilde{\pi}(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})] = \mathbb{E}_{\mathbf{z} \sim \pi(\cdot | \text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h}))} [r(\text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h}), \mathbf{z})].$$

Therefore, for every π and every such prefixed problem $\mathbf{x}_{\text{pre}} = \text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h})$,

$$\mathbb{E}_{\mathbf{z} \sim \pi(\cdot | \mathbf{x}_{\text{pre}})} [r(\mathbf{x}_{\text{pre}}, \mathbf{z})] \leq \max_{\pi' \in \Pi} \mathbb{E}_{\mathbf{y} \sim \pi'(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})]. \quad (\text{B.4})$$

Let $m(\mathbf{x})$ be the number of prefixed problems in $\mathcal{D}_{\text{pre}}$ derived from $\mathbf{x}$. Summing the above inequality over all prefixed problems and grouping by their originating $\mathbf{x}$ gives

$$\sum_{\mathbf{x}_{\text{pre}} \in \mathcal{D}_{\text{pre}}} \mathbb{E} [r(\mathbf{x}_{\text{pre}}, \cdot)] \leq \sum_{\mathbf{x} \in \mathcal{D}} m(\mathbf{x}) \max_{\pi' \in \Pi} \mathbb{E}_{\mathbf{y} \sim \pi'(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})]. \quad (\text{B.5})$$

Define the constant

$$C := \sum_{\mathbf{x} \in \mathcal{D}} m(\mathbf{x}) \max_{\pi' \in \Pi} \mathbb{E}_{\mathbf{y} \sim \pi'(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})], \quad (\text{B.6})$$

which is independent of π . Then for every $\pi \in \Pi$ we have the uniform upper bound

$$J_{\text{pre}}(\pi) \leq C + J(\pi). \quad (\text{B.7})$$

Tightness using Assumption 3.1. By realizability in Assumption 3.1, there exists a policy $\mu \in \Pi$ such that

$$\mu(\mathbf{y}^x | \mathbf{x}) = 1, \quad \forall \mathbf{x} \in \mathcal{D}. \quad (\text{B.8})$$

In particular, for any cut h , conditioning μ on the prefix $(\mathbf{y}^x)_{1:h}$ yields the deterministic continuation:

$$\mu(\mathbf{z} | \text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h})) = \mathbf{1}\{\mathbf{z} = (\mathbf{y}^x)_{h+1:|\mathbf{y}^x|}\}. \quad (\text{B.9})$$

Therefore, on every prefixed problem $\mathbf{x}_{\text{pre}} = \text{concat}(\mathbf{x}, (\mathbf{y}^x)_{1:h})$,

$$\mathbb{E}_{\mathbf{z} \sim \mu(\cdot | \mathbf{x}_{\text{pre}})} [r(\mathbf{x}_{\text{pre}}, \mathbf{z})] = r(\mathbf{x}, \mathbf{y}^x) = 1, \quad (\text{B.10})$$

where the last equality uses correctness of $\mathcal{D}_{\text{off}}$ in Assumption 3.1. Hence,

$$\sum_{\mathbf{x}_{\text{pre}} \in \mathcal{D}_{\text{pre}}} \mathbb{E}_{\mathbf{z} \sim \mu(\cdot | \mathbf{x}_{\text{pre}})} [r(\mathbf{x}_{\text{pre}}, \mathbf{z})] = \sum_{\mathbf{x} \in \mathcal{D}} m(\mathbf{x}). \quad (\text{B.11})$$

Moreover, by (B.8) and correctness,

$$J(\mu) = \sum_{\mathbf{x} \in \mathcal{D}} \mathbb{E}_{\mathbf{y} \sim \mu(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})] = \sum_{\mathbf{x} \in \mathcal{D}} r(\mathbf{x}, \mathbf{y}^x) = |\mathcal{D}|. \quad (\text{B.12})$$

Therefore $J^* = |\mathcal{D}|$ and $J(\mu) = J^*$. Since rewards are in $[0, 1]$, for each $\mathbf{x}$ we have $\max_{\pi' \in \Pi} \mathbb{E}_{\mathbf{y} \sim \pi'(\cdot | \mathbf{x})} [r(\mathbf{x}, \mathbf{y})] = 1$, and thus the constant simplifies to $C = \sum_{\mathbf{x} \in \mathcal{D}} m(\mathbf{x})$. Combining with (B.11) yields

$$J_{\text{pre}}(\mu) = C + J(\mu) = C + J^*. \quad (\text{B.13})$$

Step 3: Concluding the consistency of PrefixRL objective. Let $\hat{\pi} \in \arg\max_{\pi \in \Pi} J_{\text{pre}}(\pi)$ be any maximizer of the PrefixRL objective. By optimality of $\hat{\pi}$, (B.13), and the upper bound (B.7),

$$C + J(\hat{\pi}) \geq J_{\text{pre}}(\hat{\pi}) \geq J_{\text{pre}}(\mu) = C + J^*. \quad (\text{B.14})$$

Cancelling C yields $J(\hat{\pi}) \geq J^*$, hence $J(\hat{\pi}) = J^*$. Therefore $\hat{\pi} \in \arg\max_{\pi \in \Pi} J(\pi)$, proving that any maximizer of the PrefixRL objective also maximizes the standard RL objective.

Algorithm 1 PrefixRL with Natural Policy Gradients

Require: Base policy π^0 , off-policy data $\mathcal{D}_{\text{off}}$, horizon H , iterations T , step size η , Q function class $\mathcal{F}$.

- 1: Initialize the iterative algorithm with base policy: $\pi^1 \leftarrow \pi^0$.
- 2: **for** $t = 1, \dots, T$ **do**
- 3: Initialize dataset $\mathcal{D}_t \leftarrow \{\}$.
- 4: **for** $i = 1 \dots n$ **do**
- 5: Sample (s_h, a_h^{off}) uniformly across state-action pairs in $\mathcal{D}_{\text{off}}$. ▷ sample prefixed-problem
- 6: $a_h \leftarrow a_h^{\text{off}}$ with probability $1/2$ and $\sim \pi^t(\cdot | s_h)$ otherwise.
- 7: Execute $\pi^t(\cdot | s_h, a_h)$ from step $h+1$ through H to obtain the full trace with reward r .
- 8: $\mathcal{D}_t \leftarrow \mathcal{D}_t \cup (s_h, a_h, r)$.
- 9: **Critic fit (regression oracle):**
- 10: $\hat{Q}^t \leftarrow \operatorname{argmin}_{f \in \mathcal{F}} \sum_{(s,a,r) \in \mathcal{D}_t} (f(s,a) - r)^2$.
- 11: **Natural policy update (mirror ascent):** ▷ performed state-wise
- 12: $\pi^{t+1}(\cdot | s) \leftarrow \operatorname{argmin}_p \langle -\hat{Q}^t(s, \cdot), p \rangle + \frac{1}{\eta} \text{KL}(p \| \pi^t(\cdot | s))$.
- 13: **end for**
- 14: **end for**
- 15: **return** $\bar{\pi}_T \leftarrow \frac{1}{T} \sum_{t=1}^T \pi^t$. ▷ return mixture policy

B.2 Proof of Theorem 3.3

In this section we present our proof for Theorem 3.3 which bounds the performance suboptimality for PrefixRL. In particular, we bound this gap for an algorithm that conforms to the PrefixRL workflow (Algorithm 1) but uses natural policy gradient (NPG) (Kakade, 2001) to update the policy iteratively (starting from the base LLM π^0). In our practical implementation we use REINFORCE to compute on-policy gradients. Next we introduce the setup and describe the key steps in Algorithm 1, some differences with practice and the full proof.

Setup. We use π^0 to denote the base LLM we start RL training with, and μ as the policy used to IID sample the dataset of off-policy traces $\mathcal{D}_{\text{off}}$, one trace for each problem in $\mathbf{x} \in \mathcal{D}$. We assume that $\mathcal{D}_{\text{off}}$ is realizable (Assumption 3.1) which implies that it only consists of correct off-policy traces. Let $\mathcal{F}$ denote the class of Q -functions induced by all policies in the policy class Π , and H be the maximum token budget or horizon H of the auto-regressive Markov decision process (MDP) induced by the policies in Π and reward function r .

Description of PrefixRL with NPG (Algorithm 1). In each iteration of Algorithm 1 we *first* collect a dataset of state, action and reward triplets $\mathcal{D}_t$. *Second*, we fit a critic or Q function $\hat{Q}^t$ on this dataset of size N (step 9). *Third*, we use the fitted Q function to perform a state-wise mirror ascent or natural policy update (step 11) in order to get the subsequent RL iterate. We collect the N traces in $\mathcal{D}_t$ by uniformly sampling an off-policy state s_h (prefixed-problem) from $\mathcal{D}_{\text{off}}$. Then, we rollout the current RL policy π^t conditioned on state s_h to sample a single action or token a_h (step 6). To estimate the Q function under the current RL policy at this state-action pair we now complete the rollout till time step H and collect a reward (step 7).

Difference with practice: Algorithm 1 uses Q functions instead of direct rewards. The update in NPG is similar to REINFORCE except that we use N on-policy samples to first estimate a Q function (step 10) in the Q function class $\mathcal{F}$ for the current RL iterate π^t and then use the estimated Q function to update the policy and get π_{t+1} using mirror ascent (step 12). This is a bit different from REINFORCE where we compute the policy gradient using only the rewards attained by the N on-policy traces and perform a step of gradient.

Difference with practice: Algorithm 1 samples new prefixed-problems from $\mathcal{D}_{\text{off}}$. In practice we construct a prefixed-problems from a fixed dataset of off-policy traces $\mathcal{D}_{\text{off}}$ and also use the set of prefixed-problems in $\mathcal{D}_{\text{pre}}$ are fixed throughout RL training. In contrast, Algorithm 1 samples off-policy states (prefixed-problems) from the dataset of off-policy traces $\mathcal{D}_{\text{off}}$. This difference is pretty minor but perhaps underscores the performance improvements driven by back-generalization in being able to improve performance on the original no-prefix problems despite PrefixRL only using a small fraction of all possible off-policy prefixes in $\mathcal{D}_{\text{off}}$.

Comparison with Chang et al. (2024). Our proof technique follows Chang et al. (2024), adapting to the setting of verifiable rewards with our different “reset” policy (which we refer to as prefix policy μ), and removing the requirement of KL divergence between the current and the reset policy. Since our off-policy dataset consists

of only realizable correct traces we will need much weaker assumptions. Following are some key differences compared to Algorithm 3 in [Chang et al. \(2024\)](#) that allows us to prove the suboptimality gap with weaker assumptions. *First*, we sample the prefix from the comparator policy (in other words the prefix generating policy is realizable and lies in the class of optimal policies). This ensures sufficient coverage for the distribution of Q-function regression (ensuring small error in fitting the critic) over states visited by a “good” policy even though the current RL iterate is far from it. *Second*, we output the mixture policy (standard in self-play literature ([Bai et al., 2020](#); [Hofbauer and Sorin, 2006](#))). *Finally*, unlike [Chang et al. \(2024\)](#), we don’t require a bound on the KL divergence against the SFT policy or the policy trained on the off-policy data.

Assumptions needed for Theorem 3.3. Now, we list the assumptions we make in our analysis of the suboptimality gap of PrefixRL. In general, they are milder than the assumptions in [Chang et al. \(2024\)](#).

- Assumption B.1 is pretty standard in the analysis of actor-critic methods ([Konda and Tsitsiklis, 2002](#)) and only requires that our critic function class is expressive enough to realize the Q function induced by any policy in Π . Note that since rewards are binary and terminal the Q -value at any state $\in [0,1]$.
- Assumption B.2 is a milder form of the typical assumption on the coverage over states visited by the optimal policy. Here, we only assume that there is an optimal policy that can fit the dataset $\mathcal{D}_{\text{off}}$ we collected. Typically the coverage assumption places a uniform bound on the likelihood ratio over the state distributions of the optimal policy and the current RL policy (d^{π^*}/d^π) as in [Chang et al. \(2024\)](#).
- Assumption B.3 is necessary to ensure that the KL between the prefix generating policy (empirical distribution over $\mathcal{D}_{\text{off}}$) and the base policy π^0 is finite. If the size of the dataset $\mathcal{D}_{\text{off}} \rightarrow \infty$ and the samples in $\mathcal{D}_{\text{off}}$ are all drawn IID from a policy $\nu \in \Pi$, then this assumption requires that cross-entropy between ν and π^0 is finite.

Assumption B.1 (Realizability of Q -function class). *There is a finite Q -function class $\mathcal{F} \subseteq \{f: \mathcal{S} \times \mathcal{A} \rightarrow [0,1]\}$, and that Q -function induced by any policy is realized in this class, i.e., $Q^\pi \in \mathcal{F} \forall \pi \in \Pi$.*

Assumption B.2 (Correctness and realizability of $\mathcal{D}_{\text{off}}$). *We say that $\mathcal{D}_{\text{off}}$ is correct if it contains a single correct trace $\mathbf{y}$ for every $\mathbf{x} \in \mathcal{D}$ and realizable if $\exists$ some policy $\mu \in \Pi$ such that $\mu(\mathbf{y}|\mathbf{x}) = 1, \forall (\mathbf{x}, \mathbf{y}) \in \mathcal{D}_{\text{off}}$.*

Assumption B.3 (Bounded likelihood of $\mathcal{D}_{\text{off}}$ under π^0). *The KL divergence between base LLM π^0 and the policy $\mu \in \Pi$ that perfectly fits the data is $\text{KL}(\mu||\pi^0) < \infty$. In other words, this assumes that the samples in $\mathcal{D}_{\text{off}}$ have a bounded likelihood under the base LLM π^0 , i.e., $\text{KL}(\mu||\pi^0) = \frac{1}{|\mathcal{D}_{\text{off}}|} \sum_{(\mathbf{x}, \mathbf{y}^x) \in \mathcal{D}_{\text{off}}} -\log \pi^0(\mathbf{y}^x|\mathbf{x}) < \infty$.*

Proof. We prove the guarantee against the comparator policy μ from Assumption B.2. Since $\mathcal{D}_{\text{off}}$ is correct and realizable by μ (i.e., for each $\mathbf{x} \in \mathcal{D}$ the unique correct trace $\mathbf{y}^x$ satisfies $\mu(\mathbf{y}^x|\mathbf{x}) = 1$), we have $J(\mu) = J^*$. Thus it suffices to upper bound:

$$J(\mu) - J(\bar{\pi}_T). \quad (\text{B.15})$$

State distribution induced by $\mathcal{D}_{\text{off}}$. Let $\mathbf{s} \in \mathcal{S}$ denote an autoregressive prefix-state. Algorithm 1 samples prefix-states by drawing $(\mathbf{x}, \mathbf{y})$ uniformly from $\mathcal{D}_{\text{off}}$ and then sampling a prefix of $\mathbf{y}$ (according to the algorithm’s prefix-selection rule). Because μ deterministically generates the same trace $\mathbf{y}$ in $\mathcal{D}_{\text{off}}$ for each $\mathbf{x}$, this state distribution coincides with the state-visitation distribution of μ ; we denote it by d_s^μ .

$$d_s^\mu \equiv d_s^{\text{off}}. \quad (\text{B.16})$$

Let π^0 be the base LLM and $\{\pi^t\}_{t=1}^T$ be the iterates produced by NPG / mirror descent with stepsize η and critic $\hat{Q}^t$, and define the averaged policy:

$$\bar{\pi}_T := \frac{1}{T} \sum_{t=1}^T \pi^t. \quad (\text{B.17})$$

Performance difference lemma under d_s^μ . Applying performance difference Lemma B.4 with $(\pi, \pi') = (\mu, \pi^t)$ yields:

$$J(\mu) - J(\pi^t) = \mathbb{E}_{\mathbf{s} \sim d_s^\mu} \mathbb{E}_{\mathbf{a} \sim \mu(\cdot|\mathbf{s})} [A^{\pi^t}(\mathbf{s}, \mathbf{a})]. \quad (\text{B.18})$$

Using $A^{\pi^t}(\mathbf{s}, \mathbf{a}) = Q^{\pi^t}(\mathbf{s}, \mathbf{a}) - V^{\pi^t}(\mathbf{s})$ and the identity $\mathbb{E}_{\mathbf{a} \sim \pi^t(\cdot | \mathbf{s})}[A^{\pi^t}(\mathbf{s}, \mathbf{a})] = 0$, (B.18) can be rewritten as

$$J(\mu) - J(\pi^t) = \mathbb{E}_{\mathbf{s} \sim d_{\mathbf{s}}^{\mu}} \left[\langle Q^{\pi^t}(\mathbf{s}, \cdot), \mu(\cdot | \mathbf{s}) - \pi^t(\cdot | \mathbf{s}) \rangle \right]. \quad (\text{B.19})$$

B.2.1 Critic Estimation Error.

Fix an iteration t . The critic is fit by least squares over a finite class $\mathcal{F}$ (Assumption B.1) using N i.i.d. samples $(\mathbf{s}_k, \mathbf{a}_k, z_k)$ where $\mathbf{s}_k \sim d_{\mathbf{s}}^{\mu}$, $\mathbf{a}_k \sim \rho^t(\cdot | \mathbf{s}_k)$ (see discussion below on ρ^t), and z_k is an unbiased target for $Q^{\pi^t}(\mathbf{s}_k, \mathbf{a}_k)$. Because rewards are *terminal and binary* in $\{0, 1\}$, we have

$$0 \leq Q^{\pi^t}(\mathbf{s}, \mathbf{a}) \leq 1, \quad 0 \leq \widehat{Q}^t(\mathbf{s}, \mathbf{a}) \leq 1, \quad (\text{B.20})$$

so we may take $R=1$ in Lemma B.6. Therefore, setting $\delta_t := \delta/(2T)$ and applying Lemma B.6 with $\mathcal{H} = \mathcal{F}$, with probability at least $1 - \delta_t$,

$$\mathbb{E}_{\mathbf{s} \sim d_{\mathbf{s}}^{\mu}, \mathbf{a} \sim \rho^t(\cdot | \mathbf{s})} \left[(\widehat{Q}^t(\mathbf{s}, \mathbf{a}) - Q^{\pi^t}(\mathbf{s}, \mathbf{a}))^2 \right] \leq \frac{256}{N} \log \left(\frac{2|\mathcal{F}|}{\delta_t} \right) = \frac{256}{N} \log \left(\frac{4T|\mathcal{F}|}{\delta} \right). \quad (\text{B.21})$$

Behavior distribution ρ^t and pointwise domination. At iteration t , Algorithm 1 forms critic data by first sampling $(\mathbf{s}_h, a_h^{\text{off}})$ uniformly from $\mathcal{D}_{\text{off}}$ and then sampling

$$a_h = \begin{cases} a_h^{\text{off}} & \text{w.p. } \frac{1}{2}, \\ a_h \sim \pi^t(\cdot | \mathbf{s}_h) & \text{w.p. } \frac{1}{2}. \end{cases} \quad (\text{B.22})$$

Let $\mu(\cdot | \mathbf{s}_h)$ denote the (deterministic) conditional action distribution induced by $\mathcal{D}_{\text{off}}$, i.e.,

$$\mu(a | \mathbf{s}_h) := \mathbf{1}\{a = a_h^{\text{off}}\}. \quad (\text{B.23})$$

Then the induced action-sampling (behavior) distribution used for critic fitting is the mixture

$$\rho^t(\cdot | \mathbf{s}_h) := \frac{1}{2} \mu(\cdot | \mathbf{s}_h) + \frac{1}{2} \pi^t(\cdot | \mathbf{s}_h). \quad (\text{B.24})$$

Consequently, for every state $\mathbf{s}$ and action a , we have the pointwise lower bounds

$$\rho^t(a | \mathbf{s}) \geq \frac{1}{2} \mu(a | \mathbf{s}), \quad \rho^t(a | \mathbf{s}) \geq \frac{1}{2} \pi^t(a | \mathbf{s}), \quad (\text{B.25})$$

and hence the pointwise domination inequalities

$$\mu(a | \mathbf{s}) \leq 2\rho^t(a | \mathbf{s}), \quad \pi^t(a | \mathbf{s}) \leq 2\rho^t(a | \mathbf{s}). \quad (\text{B.26})$$

In particular, (B.25) also implies absolute continuity: if $\rho^t(a | \mathbf{s}) = 0$ then $\mu(a | \mathbf{s}) = \pi^t(a | \mathbf{s}) = 0$, so $\mu(\cdot | \mathbf{s}) \ll \rho^t(\cdot | \mathbf{s})$ and $\pi^t(\cdot | \mathbf{s}) \ll \rho^t(\cdot | \mathbf{s})$.

Then Cauchy–Schwarz and Jensen applied to (B.21) yield, for $\pi \in \{\mu, \pi^t\}$,

$$\left| \mathbb{E}_{\mathbf{s} \sim d_{\mathbf{s}}^{\mu}, \mathbf{a} \sim \pi(\cdot | \mathbf{s})} [\widehat{Q}^t(\mathbf{s}, \mathbf{a}) - Q^{\pi^t}(\mathbf{s}, \mathbf{a})] \right| \leq \epsilon_{\text{crt}}, \quad \epsilon_{\text{crt}} := 16\sqrt{2} \sqrt{\frac{1}{N} \log \left(\frac{4T|\mathcal{F}|}{\delta} \right)}. \quad (\text{B.27})$$

Taking a union bound over $t \in [T]$, with probability at least $1 - \delta/2$, (B.27) holds for all t .

B.2.2 Mirror Ascent and NPG Optimization Error.

The mirror-descent update at state $\mathbf{s}$ is the KL-regularized maximization

$$\pi^{t+1}(\cdot | \mathbf{s}) = \arg \max_{p(\cdot | \mathbf{s})} \left\{ \eta \langle \widehat{Q}^t(\mathbf{s}, \cdot), p(\cdot | \mathbf{s}) \rangle - \text{KL}(p(\cdot | \mathbf{s}) \| \pi^t(\cdot | \mathbf{s})) \right\}.$$

The first-order optimality condition implies that for any $p(\cdot | \mathbf{s})$,

$$\left\langle -\eta \widehat{Q}^t(\mathbf{s}, \cdot) + \nabla_r \text{KL}(r(\cdot | \mathbf{s}) \| \pi^t(\cdot | \mathbf{s})) \Big|_{r=\pi^{t+1}}, p - \pi^{t+1} \right\rangle \geq 0. \quad (\text{B.28})$$

Set $p = \mu(\cdot | \mathbf{s})$ and rearrange (B.28) to obtain

$$\eta \langle \widehat{Q}^t(\mathbf{s}, \cdot), \mu(\cdot | \mathbf{s}) - \pi^{t+1}(\cdot | \mathbf{s}) \rangle \leq \left\langle \nabla_r \text{KL}(r(\cdot | \mathbf{s}) \| \pi^t(\cdot | \mathbf{s})) \Big|_{r=\pi^{t+1}}, \mu - \pi^{t+1} \right\rangle. \quad (\text{B.29})$$

Apply the KL three-point identity (Lemma (B.62)) with $p = \mu(\cdot | \mathbf{s})$, $r = \pi^{t+1}(\cdot | \mathbf{s})$, and $q = \pi^t(\cdot | \mathbf{s})$ to rewrite the right-hand side:

$$\left\langle \nabla_r \text{KL}(r(\cdot | \mathbf{s}) \| \pi^t(\cdot | \mathbf{s})) \Big|_{r=\pi^{t+1}}, \mu - \pi^{t+1} \right\rangle = \text{KL}(\mu \| \pi^t) - \text{KL}(\mu \| \pi^{t+1}) - \text{KL}(\pi^{t+1} \| \pi^t). \quad (\text{B.30})$$

Combining (B.29) and (B.30) gives

$$\eta \langle \widehat{Q}^t(\mathbf{s}, \cdot), \mu - \pi^{t+1} \rangle \leq \text{KL}(\mu \| \pi^t) - \text{KL}(\mu \| \pi^{t+1}) - \text{KL}(\pi^{t+1} \| \pi^t). \quad (\text{B.31})$$

Using (B.20) and Pinsker's inequality, we bound the shift term

$$\eta \langle \widehat{Q}^t(\mathbf{s}, \cdot), \pi^{t+1} - \pi^t \rangle \leq \eta \|\widehat{Q}^t(\mathbf{s}, \cdot)\|_\infty \|\pi^{t+1} - \pi^t\|_1 \leq \frac{\eta^2}{2} + \text{KL}(\pi^{t+1} \| \pi^t), \quad (\text{B.32})$$

where the last inequality uses $\|\widehat{Q}^t\|_\infty \leq 1$ and $\|\pi^{t+1} - \pi^t\|_1^2 \leq 2\text{KL}(\pi^{t+1} \| \pi^t)$. Adding (B.31) and (B.32) cancels $\text{KL}(\pi^{t+1} \| \pi^t)$ and yields

$$\eta \langle \widehat{Q}^t(\mathbf{s}, \cdot), \mu - \pi^t \rangle \leq \text{KL}(\mu \| \pi^t) - \text{KL}(\mu \| \pi^{t+1}) + \frac{\eta^2}{2}. \quad (\text{B.33})$$

Taking expectation over $\mathbf{s} \sim d_s^\mu$ and summing over $t = 1, \dots, T$ gives

$$\frac{1}{T} \sum_{t=1}^T \mathbb{E}_{\mathbf{s} \sim d_s^\mu} \left[\langle \widehat{Q}^t(\mathbf{s}, \cdot), \mu - \pi^t \rangle \right] \leq \frac{D_1}{\eta T} + \frac{\eta}{2}, \quad (\text{B.34})$$

where

$$D_1 := \mathbb{E}_{\mathbf{s} \sim d_s^\mu} \left[\text{KL}(\mu(\cdot | \mathbf{s}) \| \pi^1(\cdot | \mathbf{s})) \right]. \quad (\text{B.35})$$

Using $\pi^1 = \pi^0$ and the definition of $\text{KL}(\mu \| \pi^0)$ under the $\mathcal{D}_{\text{off}}$ -induced state distribution (Assumption B.3), we identify

$$D_1 = \text{KL}(\mu \| \pi^0). \quad (\text{B.36})$$

B.2.3 Combining Critic Error and Optimization Error.

Starting from (B.19), add and subtract $\widehat{Q}^t$:

$$\begin{aligned} J(\mu) - J(\pi^t) &= \mathbb{E}_{\mathbf{s} \sim d_s^\mu} \left[\langle \widehat{Q}^t(\mathbf{s}, \cdot), \mu - \pi^t \rangle \right] \\ &\quad + \mathbb{E}_{\mathbf{s} \sim d_s^\mu} \left[\langle Q^{\pi^t}(\mathbf{s}, \cdot) - \widehat{Q}^t(\mathbf{s}, \cdot), \mu - \pi^t \rangle \right]. \end{aligned} \quad (\text{B.37})$$

On the high-probability event where (B.27) holds for both $\pi = \mu$ and $\pi = \pi^t$, the critic-error term is bounded by

$$\mathbb{E}_{\mathbf{s} \sim d_s^\mu} \left[\langle Q^{\pi^t} - \widehat{Q}^t, \mu - \pi^t \rangle \right] \leq \left| \mathbb{E}_{\mathbf{s} \sim d_s^\mu, \mathbf{a} \sim \mu} [\widehat{Q}^t - Q^{\pi^t}] \right| + \left| \mathbb{E}_{\mathbf{s} \sim d_s^\mu, \mathbf{a} \sim \pi^t} [\widehat{Q}^t - Q^{\pi^t}] \right| \leq 2\epsilon_{\text{crt}}, \quad (\text{B.38})$$

which introduces no extra factor of H .

Averaging (B.37) over $t=1,\dots,T$ and using (B.17), we obtain

$$J(\mu) - J(\bar{\pi}_T) \leq \frac{1}{T} \sum_{t=1}^T \mathbb{E}_{\mathbf{s} \sim d_s^\mu} \left[\langle \hat{Q}^t(\mathbf{s}, \cdot), \mu - \pi^t \rangle \right] + 2\epsilon_{\text{crt}}. \quad (\text{B.39})$$

Applying (B.34) to the first term in (B.39) yields

$$J(\mu) - J(\bar{\pi}_T) \leq \frac{D_1}{\eta T} + \frac{\eta}{2} + 2\epsilon_{\text{crt}}. \quad (\text{B.40})$$

Choose $\eta := \sqrt{\frac{2D_1}{T}}$ to balance the first two terms in (B.40), giving

$$\frac{D_1}{\eta T} + \frac{\eta}{2} = \sqrt{\frac{2D_1}{T}}. \quad (\text{B.41})$$

Combining (B.36), (B.27), and (B.40), and recalling $J(\mu) = J^*$, we conclude that with probability at least $1 - \delta$,

$$J(\pi^*) - J(\bar{\pi}_T) = J^* - J(\bar{\pi}_T) \leq \mathcal{O} \left(\sqrt{\frac{\text{KL}(\mu \parallel \pi^0)}{T}} + \sqrt{\frac{1}{N} \log \left(\frac{T|\mathcal{F}|}{\delta} \right)} \right). \quad (\text{B.42})$$

□

B.3 Proof of Proposition 3.4

Now, we prove our separation result in Proposition 3.4 that lower bounds the performance gap between standard RL and PrefixRL. Here, standard RL runs Algorithm 1 but now without any access to the off-policy dataset $\mathcal{D}_{\text{off}}$. In each iteration, the critic training dataset $\mathcal{D}_t$ is now populated with $(\mathbf{s}, a, r)$ tuples where both $\mathbf{s}$ and a are sampled from the current policy π^t . So, unlike PrefixRL we never sample the state or prefix from $\mathcal{D}_{\text{off}}$. In this simple worst-case instance we present below there is a single trajectory in $\mathcal{D}_{\text{off}}$ that is also what that the optimal policy samples with probability 1 and attains performance of $J(\pi^*) = 1$.

Proof. We present (i) the MDP instance together with a choice of *base policy* that generates the off-policy trace, and then (ii) an exponential lower bound for standard on-policy RL without $\mathcal{D}_{\text{off}}$, and (iii) a horizon-independent (non-exponential) upper bound for PrefixRL with $\mathcal{D}_{\text{off}}$.

MDP instance (hidden rewarding binary string) and base policy. Fix a horizon H and an unknown binary string $\mathbf{b} = (b_1, \dots, b_H) \in \{0, 1\}^H$. Let the state space be

$$\mathcal{S} = \{s_0, s_1, \dots, s_H\}, \quad (\text{B.43})$$

where s_{h-1} encodes the first $h-1$ actions taken so far (s_0 is the start state). The action space is $\mathcal{A} = \{0, 1\}$. Transitions are deterministic: from s_{h-1} , taking action $a_h \in \{0, 1\}$ moves to s_h . The episode ends at s_H with terminal reward

$$r = \mathbf{1}\{(a_1, \dots, a_H) = (b_1, \dots, b_H)\}. \quad (\text{B.44})$$

Thus, exactly one length- H action sequence earns reward 1.

Let π^* be the deterministic policy that selects b_h at s_{h-1} for each $h \in [H]$. Then $J(\pi^*) = 1$. For the PrefixRL part, we also choose a *base policy* μ and an off-policy dataset $\mathcal{D}_{\text{off}}$: we set $\mu := \pi^*$ and let $\mathcal{D}_{\text{off}}$ contain the unique successful trajectory of μ , equivalently the state-action pairs

$$\mathcal{D}_{\text{off}} = \{(s_{h-1}, b_h) : h \in [H]\}. \quad (\text{B.45})$$

B.3.1 Exponential lower bound for standard on-policy RL (Algorithm 1 without $\mathcal{D}_{\text{off}}$).

We analyze an on-policy variant of Algorithm 1 in which *there is no off-policy dataset*: at each iteration t , the algorithm samples N full episodes only from its current policy π^t , observes only terminal rewards $r^{(t,i)} \in \{0,1\}$, fits a critic, and updates the policy. Let $\hat{\pi}_T$ denote the (possibly randomized) policy output after T iterations (so the total number of full episodes is TN). We prove that for any such algorithm, there exists an instance $\mathbf{b}$ for which the expected suboptimality gap is at least $1 - (TN+1)2^{-H}$.

Yao's minimax setup. By Yao's minimax principle [Yao \(1977\)](#), it suffices to fix an arbitrary adaptive algorithm and analyze its expected performance when the instance is random:

$$\mathbf{b} \sim \text{Unif}(\{0,1\}^H), \quad (\text{B.46})$$

and we write $\mathbb{P}_{\mathbf{b}}, \mathbb{E}_{\mathbf{b}}$ for probability/expectation over this draw (and over the algorithm's internal randomness).

Per-rollout success probability is 2^{-H} . Fix any rollout index (t,i) . Condition on the full interaction history up to this rollout and on the algorithm's internal randomness. Under this conditioning, the action string $\mathbf{a}^{(t,i)} \in \{0,1\}^H$ is some random element (with an arbitrary distribution induced by the algorithm), while $\mathbf{b}$ remains uniform and independent. Therefore,

$$\mathbb{P}_{\mathbf{b}}[r^{(t,i)} = 1 \mid \text{history}] = \mathbb{P}_{\mathbf{b}}[\mathbf{a}^{(t,i)} = \mathbf{b} \mid \text{history}] = \sum_{\mathbf{a} \in \{0,1\}^H} \mathbb{P}[\mathbf{a}^{(t,i)} = \mathbf{a} \mid \text{history}] \cdot \mathbb{P}_{\mathbf{b}}[\mathbf{b} = \mathbf{a}] = 2^{-H}. \quad (\text{B.47})$$

Taking expectation over the history yields the unconditional version:

$$\mathbb{P}_{\mathbf{b}}[r^{(t,i)} = 1] = 2^{-H}. \quad (\text{B.48})$$

Probability of ever seeing a reward-1 rollout. There are exactly TN rollouts in total. By a union bound and (B.48) we get the following upper bound on the probability of seeing a reward 1 rollout across all T steps,

$$\mathbb{P}_{\mathbf{b}}[\exists t \leq T, i \leq N : r^{(t,i)} = 1] \leq \sum_{t=1}^T \sum_{i=1}^N \mathbb{P}_{\mathbf{b}}[r^{(t,i)} = 1] = TN \cdot 2^{-H}. \quad (\text{B.49})$$

Bound the expected value of the returned policy. Let $\mathbf{a} \sim \hat{\pi}_T$ denote the length- H string generated by rolling out $\hat{\pi}_T$ from s_0 . On instance $\mathbf{b}$, $J(\hat{\pi}_T) = \mathbb{P}[\mathbf{a} = \mathbf{b}]$. We can decompose this reward on two events: whether the training interaction ever produced a reward-1 rollout or not:

$$\mathbb{E}_{\mathbf{b}}[J(\hat{\pi}_T)] \leq \mathbb{P}_{\mathbf{b}}[\exists t, i : r^{(t,i)} = 1] \cdot 1 + \mathbb{P}_{\mathbf{b}}[\forall t, i : r^{(t,i)} = 0] \cdot \sup_{\hat{\pi}_T} \mathbb{E}_{\mathbf{b}}[J(\hat{\pi}_T) \mid \forall t, i : r^{(t,i)} = 0]. \quad (\text{B.50})$$

On the event $\{\forall t, i : r^{(t,i)} = 0\}$, the algorithm has never observed the rewarding string. Each zero-reward rollout rules out *at most one* candidate string, namely the realized action string $\mathbf{a}^{(t,i)}$. Hence, conditioning on $\{\forall t, i : r^{(t,i)} = 0\}$ only implies that $\mathbf{b}$ is not in a set of at most TN excluded strings. Under the prior $\mathbf{b} \sim \text{Unif}(\{0,1\}^H)$, the posterior is uniform over the remaining candidates, so

$$\max_{\mathbf{a} \in \{0,1\}^H} \mathbb{P}_{\mathbf{b}}[\mathbf{b} = \mathbf{a} \mid \forall t, i : r^{(t,i)} = 0] \leq \frac{1}{2^H - TN}, \quad (\text{B.51})$$

and therefore

$$\sup_{\hat{\pi}_T} \mathbb{E}_{\mathbf{b}}[J(\hat{\pi}_T) \mid \forall t, i : r^{(t,i)} = 0] = \sup_{\hat{\pi}_T} \mathbb{E}_{\mathbf{b}}[\mathbb{P}(\mathbf{a} = \mathbf{b} \mid \hat{\pi}_T, \forall t, i : r^{(t,i)} = 0)] \leq \frac{1}{2^H - TN}. \quad (\text{B.52})$$

Substituting (B.49) and (B.52) into (B.50) yields

$$\mathbb{E}_{\mathbf{b}}[J(\hat{\pi}_T)] \leq \mathbb{P}_{\mathbf{b}}[\exists t, i : r^{(t,i)} = 1] \cdot 1 + \mathbb{P}_{\mathbf{b}}[\forall t, i : r^{(t,i)} = 0] \cdot \frac{1}{2^H - TN} \leq TN \cdot 2^{-H} + \frac{1}{2^H - TN}. \quad (\text{B.53})$$

In particular, whenever $TN \leq 2^{H-1}$ we have $\frac{1}{2^H - TN} \leq 2^{-(H-1)}$, and thus

$$\mathbb{E}_{\mathbf{b}}[J(\hat{\pi}_T)] \leq TN \cdot 2^{-H} + 2^{-(H-1)} \leq (TN+2)2^{-(H-1)}. \quad (\text{B.54})$$

Suboptimality lower bound and fix an instance. Since $J(\pi^*) = 1$, we obtain from (B.53) the gap bound

$$\mathbb{E}_{\mathbf{b}}[J(\pi^*) - J(\hat{\pi}_T)] \geq 1 - (TN+2)2^{-(H-1)}. \quad (\text{B.55})$$

By Yao’s minimax principle (Yao, 1977), there exists a fixed hidden string $\mathbf{b}$ for which the same bound holds for the algorithm on that instance. This proves the exponential lower bound for standard on-policy RL without access to $\mathcal{D}_{\text{off}}$.

B.3.2 Horizon-independent (non-exponential) upper bound for PrefixRL.

We now analyze Algorithm 1 on the same instance *with* $\mathcal{D}_{\text{off}}$. As discussed, the separation mechanism is that Algorithm 1 explicitly samples states from $\mathcal{D}_{\text{off}}$, which are precisely the prefix states along the π^* trajectory, thereby forcing visitation of optimal-trajectory states.

Choosing π^0 . Let π^0 denote the initialization policy of Algorithm 1, *i.e.* $\pi^1 = \pi^0$. We choose π^0 to be the uniform policy on $\{0,1\}$ at every state (*crucially, π^0 is not instance dependent*):

$$\pi_0(0|s) = \pi_0(1|s) = \frac{1}{2} \quad \text{for all } s \in \{s_0, \dots, s_{H-1}\}. \quad (\text{B.56})$$

Since $\mu = \pi^*$ is deterministic on the states in $\mathcal{D}_{\text{off}}$, the KL term in Theorem 3.1 is not exponential in H . Under the state-averaged convention used in Theorem 3.1,

$$\text{KL}(\mu \parallel \pi_0) := \mathbb{E}_{s \sim d_s^\mu} \left[\text{KL}(\mu(\cdot|s) \parallel \pi_0(\cdot|s)) \right] = \log 2, \quad (\text{B.57})$$

while under the summed convention it is $H \log 2$; in either case it is not exponential in H .

Invoking PrefixRL guarantee in Theorem 3.3. All assumptions required by Theorem 3.3 hold on this instance with $\mu = \pi^*$ and the corresponding $\mathcal{D}_{\text{off}} = \{\mathbf{b}\}$. Therefore, applying Theorem 3.3 to Algorithm 1 yields (with probability at least $1 - \delta$) the bound

$$J(\mu) - J(\bar{\pi}_T) \leq \sqrt{\frac{2\text{KL}(\mu \parallel \pi_0)}{T}} + \mathcal{O}\left(\sqrt{\frac{1}{N} \log\left(\frac{T|\mathcal{F}|}{\delta}\right)}\right), \quad (\text{B.58})$$

where $\bar{\pi}_T$ is the iterate-averaged policy output by Algorithm 1. Since $\mu = \pi^*$ on this instance, $J(\mu) = J(\pi^*) = 1$, and (B.58) implies a non-exponential suboptimality bound for PrefixRL. In particular, with the choice (B.56) we have $\text{KL}(\mu \parallel \pi_0) = \log 2$ (or $H \log 2$ under the summed convention), so the bound has no 2^{-H} term.

Separation mechanism. The on-policy exponential lower bound arises because, without $\mathcal{D}_{\text{off}}$, an algorithm only observes nonzero reward if it guesses the entire length- H string correctly in a single episode. In contrast, Algorithm 1 with $\mathcal{D}_{\text{off}}$ repeatedly samples *states along the π^* trajectory* via $\sim \mathcal{D}_{\text{off}}$ and trains at those states using the mixture distribution of Algorithm 1. Overall, this yields a non-exponential sample complexity, establishing a worst-case separation.

B.3.3 Worst-Case Separation Result Between Standard RL and PrefixRL.

The analysis in the above subsection can be viewed either for a fixed hidden string $\mathbf{b}$, or under a randomized instance distribution. In particular, let $\mathbf{b} \sim \text{Unif}(\{0,1\}^H)$, and note that under this randomization the off-policy dataset $\mathcal{D}_{\text{off}} = \mathcal{D}_{\text{off}}(\mathbf{b})$ also changes with $\mathbf{b}$ since it contains the unique successful trajectory $(s_{h-1}, b_h)_{h=1}^H$.

Lower bound for standard on-policy RL under random $\mathbf{b}$. For any on-policy algorithm that runs for T iterations with N full episodes per iteration (so TN total episodes) and does not have access to $\mathcal{D}_{\text{off}}$, the expected value of the suboptimality gap satisfies

$$\mathbb{E}_{\mathbf{b}}[J(\pi_{\mathbf{b}}^*) - J(\hat{\pi}_T)] \geq 1 - (TN+2)2^{-(H-1)}, \quad (\text{B.59})$$

where $\pi_{\mathbf{b}}^*$ denotes the optimal policy for instance $\mathbf{b}$ and $TN \leq 2^{H-1}$.

Upper bound for PrefixRL under random $\mathbf{b}$. Now consider Algorithm 1 (PrefixRL) run on the same randomized instance, where $\mathcal{D}_{\text{off}} = \mathcal{D}_{\text{off}}(\mathbf{b})$ is provided to the algorithm. Choose the initialization π_0 to be the uniform policy (so that $\text{KL}(\mu_{\mathbf{b}} \parallel \pi_0)$ is not exponential in H), with $\mu_{\mathbf{b}} := \pi_{\mathbf{b}}^*$ as the base policy that generates $\mathcal{D}_{\text{off}}(\mathbf{b})$. Invoking the previously proved Theorem 3.1 yields (with probability at least $1 - \delta$ over the algorithm's sampling)

$$\mathbb{E}_{\mathbf{b}}[J(\pi_{\mathbf{b}}^*) - J(\bar{\pi}_T)] \leq \sqrt{\frac{2\mathbb{E}_{\mathbf{b}}[\text{KL}(\mu_{\mathbf{b}} \parallel \pi_0)]}{T}} + \mathcal{O}\left(\sqrt{\frac{1}{N} \log\left(\frac{T|\mathcal{F}|}{\delta}\right)}\right), \quad (\text{B.60})$$

and for the uniform initialization π_0 we have $\mathbb{E}_{\mathbf{b}}[\text{KL}(\mu_{\mathbf{b}} \parallel \pi_0)] = \log 2$ under the state-averaged convention (or $H \log 2$ under the summed convention), which is not exponential in H .

Applying Yao's minimax principle. Equations (B.59) and (B.60) establish an *average-case* separation under the uniform distribution over instances $\mathbf{b}$ (with $\mathcal{D}_{\text{off}}$ coupled to $\mathbf{b}$ in the PrefixRL case). By Yao's minimax principle, this implies that there exists a fixed instance $\mathbf{b}$ (and the above choice of initialization π_0 , *e.g.* uniform) for which the same separation holds for any randomized standard on-policy algorithm without $\mathcal{D}_{\text{off}}$ versus Algorithm 1 with $\mathcal{D}_{\text{off}}$. $\square$

B.4 Auxiliary Lemmas

Lemma B.4 (Performance difference lemma; (Kakade and Langford, 2002)). *For all policies π, π' and initial state distribution ρ ,*

$$\mathbb{E}_{\mathbf{s}_0 \sim \rho} [V^{\pi}(\mathbf{s}_0) - V^{\pi'}(\mathbf{s}_0)] = \mathbb{E}_{\mathbf{s}_h \sim d_{\mathbf{s}}^{\pi}} \mathbb{E}_{a_h \sim \pi(\cdot | \mathbf{s}_h)} [A^{\pi'}(\mathbf{s}_h, a_h)]. \quad (\text{B.61})$$

Proof. See proof of Lemma 6.1 in Kakade and Langford (2002). $\square$

Lemma B.5 (Three-point identity for KL). *Let p, q, r be distributions on a common measurable space such that $p \ll r \ll q$ and all quantities below are finite. Then*

$$\text{KL}(p \parallel q) = \text{KL}(p \parallel r) + \text{KL}(r \parallel q) + \langle p - r, \nabla_r \text{KL}(r \parallel q) \rangle, \quad (\text{B.62})$$

. *For discrete distributions p, q, r we have:*

$$\nabla_r \text{KL}(r \parallel q)(x) = \log \frac{r(x)}{q(x)} + 1. \quad (\text{B.63})$$

Equivalently we can state this the three-point identity for discrete distributions as:

$$\text{KL}(p \parallel q) = \text{KL}(p \parallel r) + \text{KL}(r \parallel q) + \left\langle p - r, \log \frac{r}{q} \right\rangle, \quad (\text{B.64})$$

since $\langle p - r, 1 \rangle = \int (p - r) = 0$ (or $\sum_i (p_i - r_i) = 0$).

Proof. We work in the continuous case; the discrete case is identical with integrals replaced by sums. Recall

$$\text{KL}(a \parallel b) = \int a(x) \log \frac{a(x)}{b(x)} dx.$$

Consider the difference between the left-hand side and the first two KL terms:

$$\begin{aligned}
\text{KL}(p\|q) - \text{KL}(p\|r) - \text{KL}(r\|q) &= \int p \log \frac{p}{q} dx - \int p \log \frac{p}{r} dx - \int r \log \frac{r}{q} dx \\
&= \int p \left(\log \frac{p}{q} - \log \frac{p}{r} \right) dx - \int r \log \frac{r}{q} dx \\
&= \int p \log \frac{r}{q} dx - \int r \log \frac{r}{q} dx \\
&= \int (p-r) \log \frac{r}{q} dx \\
&= \left\langle p-r, \log \frac{r}{q} \right\rangle.
\end{aligned}$$

This proves (B.64). To obtain the gradient form (B.62), note that for $\text{KL}(r\|q) = \int r \log(r/q) dx$ the pointwise functional derivative with respect to r is

$$\nabla_r \text{KL}(r\|q)(x) = \log \frac{r(x)}{q(x)} + 1,$$

so

$$\langle p-r, \nabla_r \text{KL}(r\|q) \rangle = \left\langle p-r, \log \frac{r}{q} \right\rangle + \langle p-r, 1 \rangle.$$

Finally, $\langle p-r, 1 \rangle = \int (p-r) dx = 1 - 1 = 0$, yielding (B.62). $\square$

Lemma B.6 (Lemma 15 in Song et al. (2022)). *Fix any $R > 0$, $\delta \in (0, 1)$, and assume we have a class of real-valued functions $\mathcal{H} : \mathcal{X} \rightarrow [-R, R]$. Suppose we have K i.i.d. samples $\{(x_k, y_k)\}_{k=1}^K$ where $x_k \sim \rho$ and y_k is sampled via the conditional probability $p(\cdot | x_k)$:*

$$y_k \sim p(\cdot | x_k) := h^*(x_k) + \epsilon_k,$$

where $h^* \in \mathcal{H}$ and $\{\epsilon_k\}_{k=1}^K$ are independent random variables such that $\mathbb{E}[y_k | x_k] = h^*(x_k)$. Additionally, suppose that $\max_k |y_k| \leq R$ and $\max_x |h^*(x)| \leq R$. Then the least squares solution

$$\hat{h} \leftarrow \underset{h \in \mathcal{H}}{\operatorname{argmin}} \sum_{k=1}^K (h(x_k) - y_k)^2$$

satisfies, with probability at least $1 - \delta$,

$$\mathbb{E}_{x \sim \rho} \left[(\hat{h}(x) - h^*(x))^2 \right] \leq \frac{256 R^2 \log(2|\mathcal{H}|/\delta)}{K}.$$

The proof is the same as in Song et al. (2022) and thus is omitted here.

C Additional Experiment Details and Results on Back-Generalization

This appendix provides (i) the concrete hard problems used in the in-context back-generalization experiment in Section 4, and (ii) an additional train test mismatch study where the off-policy prefixes are sourced from a different model family (Figure 14).

C.1 Hard problems used in the in-context back-generalization experiment

Section 4 studies a meta-learning style setting where the policy is trained on a *prefixed problem* consisting of an in-context example (a full solved hard problem) followed by a target hard problem. We compare transfer when

the in-context hard problem is structurally similar to the target problem versus when it is unrelated (Figure ??). We use the following three problems:

Hard Problem P1 (Pass@16 for base model is 0.119 and is different from P3)

A sphere tangent to the xy -plane has center having z -coordinate > 0 . If it is projected from $P = (0, b, a)$ to the xy -plane, it gives the conic section $y = x^2$. If $a = p/q$ for integers p, q what is $p+q$? Answer: 3.

Hard Problem P2 (Pass@16 for base model is 0.074 and is similar to P3)

League has 30 teams (East 16, West 14). Inside each division everyone has played others once. If we add interleague games, what is smallest k for which every team gets exactly k games? Answer: 29.

Hard Problem P3 (Pass@16 for base model is 0.063)

Amongst 300 people, no one has more than $k-1$ friends. What is the smallest k for which it might be impossible to create some new friendships so that everyone has exactly k friends? Answer: 151.

Relatedness criterion (P2 is similar to P3; P1 is different from P3). P2 and P3 are both naturally expressed as graph feasibility problems with degree constraints, and their solutions rely on reasoning about global constraints induced by local degree requirements (regularity, parity, and obstruction arguments). In contrast, P1 is a geometry and conic projection problem whose solution structure does not share this graph-theoretic scaffold. Figure ?? uses this controlled notion of similarity to separate two effects: when the prefix and suffix share a compatible representation (P2→P3), training on prefixed problems yields substantially stronger transfer than when they do not (P1→P3).

C.2 Back-generalization under model-family mismatch

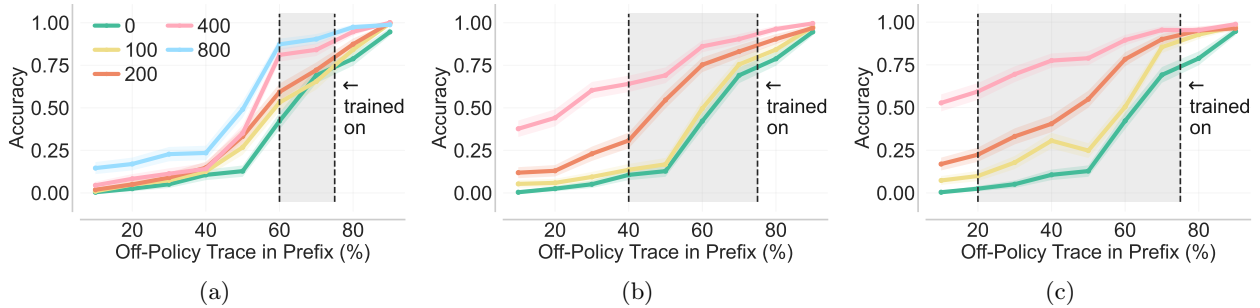


Figure 14 Back-generalization (train-test mismatch): On Llama3.1-8b-instruct we run RL only on prefixed problems sourced from Qwen3-4b-instruct whose prefix length (percent of the off-policy trace) lies in the shaded interval. At evaluation, we test across the full range of prefix lengths, including no-prefix problems (0% prefix). The performance at different RL training iterations (0, 100, 200, 400 and 800) is represented with different colors. Similar to Figure 5, where the prefixes are also sourced from Llama-3.1-8b When the mismatch is moderate, training on longer prefixes improves performance on shorter prefixes and can eventually improve no-prefix, indicating back-generalization (b, c). But different from Figure 5, we find that when the prefix distribution is skewed towards long prefixes back-generalization is weaker despite running RL for 800 iterations.

This section reports an additional train test mismatch experiment analogous to Figure 5, but where the off-policy prefixes used to form prefixed problems are sourced from a different model family. Concretely, we train Llama3.1-8B-Instruct with PrefixRL while constructing prefixed problems from correct off-policy traces generated by Qwen3-4B-Instruct.

Setup. We first collect a pool of correct off-policy traces from Qwen3-4B-Instruct. Each trace induces a family of *prefix states* by truncating the trace to a specified prefix length (reported as a percentage of the full trace). A prefixed problem is formed by conditioning the training policy on such a prefix state and then asking it to complete the solution for the same underlying problem. During training, we restrict prefix lengths to lie in a band (the shaded interval in Figure 14), and we run on-policy RL *only* on these prefixed problems. At evaluation, we sweep prefix lengths across the full range, including the no-prefix endpoint (0% prefix), and report accuracy at multiple RL training iterations.

Cross-family prefixes can still induce back-generalization. Figure 14 shows that cross-family prefixes can still induce back-generalization, but the effect is less robust than in the same-family setting of Figure 5. When the mismatch is moderate (training includes prefixes that are not concentrated exclusively at the very end of traces), improvements appear near the trained prefix region and then propagate to shorter prefixes, eventually improving no-prefix performance (Figure 14b,c). In contrast, when the training prefix distribution is skewed toward long prefixes, back-generalization is weaker, and the transfer to shorter prefixes and to no-prefix remains limited even after long training (800 iterations) (Figure 14a).

Choosing the right prefix distribution. A key difficulty in this setting is that the training prefixes are not sampled from the current policy. They are injected from an external model (Qwen), and therefore correspond to states that can be very unlikely under the evolving Llama policy. When training concentrates on very late prefixes, the policy can improve primarily on a narrow slice of the prefixed state distribution without sufficiently shaping behavior on earlier states that dominate no-prefix rollouts. This makes cross-family prefixing most effective when the training mixture covers a sufficiently broad range of prefix lengths, rather than concentrating only on long, heavily conditioned prefixes.

D Additional Experiments and Details for Results in Section 5

D.1 Off-Policy Token-Reweighting Baselines

In our work, we use CISPO (Chen et al., 2025a) to compute the policy gradient following Khatri et al. (2025) which found it to work best on long RL runs. Moreover, CISPO can also handle off-policy updates, *i.e.*, update π^t on a trajectory τ sampled from $\mu \neq \pi^t$ with an importance weighting term common in off-policy RL (Fujimoto et al., 2018). For each $\mathbf{x}$ in batch $\mathcal{B}$, CISPO samples k responses $\{\mathbf{y}_i^x\}_{i=1}^k$ where $\mathbf{y}_i^x \sim \mu(\cdot | \mathbf{x})$, then the CISPO policy gradient is given by:

$$\begin{aligned} \text{(CISPO)} \quad & \frac{1}{\text{token-sum}} \sum_{\mathbf{x} \in \mathcal{B}} \sum_{i=1}^k \sum_{h=1}^{|\mathbf{y}_i^x|} (\text{stop-grad}(\max(w(\mathbf{x}, \mathbf{y}_{i,h}^x), \varepsilon_{\text{high}})) \cdot A(\mathbf{x}, \mathbf{y}_i^x) \cdot \nabla_{\pi} \log \pi^t(\mathbf{y}_{i,h}^x | \mathbf{x}, \mathbf{y}_{i,h}^x)), \\ & \text{where } w(\mathbf{x}, \mathbf{y}_{i,h}^x) = \frac{\pi^t(\mathbf{y}_{i,h}^x | \mathbf{x}, \mathbf{y}_{i,<h}^x)}{\mu(\mathbf{y}_{i,h}^x | \mathbf{x}, \mathbf{y}_{i,<h}^x)} \quad \text{and} \quad \text{token-sum} = \sum_{\mathbf{x} \in \mathcal{B}} \sum_{i \in [k]} |\mathbf{y}_i^x|. \end{aligned} \quad (\text{D.1})$$

In (D.1), the advantage $A(\mathbf{x}, \mathbf{y}_i^x)$ is computed by subtracting the baseline $\bar{r}(x) = \frac{1}{k} \sum_{i=1}^k r(\mathbf{x}, \mathbf{y}_i^x)$ from $r(\mathbf{x}, \mathbf{y}_i^x)$. The per-token importance weight $w(\mathbf{x}, \mathbf{y}_{i,h}^x)$ accounts for the distribution shift between the current policy π^t and the sampler μ and is 1 for on-policy traces where $\pi^t = \mu$. To reduce gradient variance from importance weights, it is clipped at $\varepsilon_{\text{high}}$.

D.2 Qwen with Llama Prefixes

Figure 15 complements the cross-family back-generalization results in Figure 14 by reversing the direction of prefix transfer: instead of training Llama using Qwen-sourced off-policy prefixes, we train Qwen using prefixes sourced from Llama. The key takeaway is asymmetric transfer. While Qwen prefixes can still drive back-generalization when optimizing Llama (Section C.2), Llama prefixes are noticeably less effective for improving Qwen in both training accuracy (no-prefix training problems) and standardized evaluation (AIME 2025).

This asymmetry is consistent with the back-generalization discussion: PrefixRL relies on prefix states that are injected from an external distribution, and the degree to which learning transfers to no-prefix behavior depends on how informative and “compatible” those prefix states are with the target model’s internal representations and solution strategies. When prefixes are sourced from a more capable model family (here, Qwen), they tend to encode higher-quality intermediate reasoning states, and RL can more readily propagate improvements from prefixed states to earlier states and ultimately to the no-prefix setting. In contrast, prefixes sourced from a less capable model (here, Llama) are both (i) harder to obtain via rejection sampling and (ii) less likely to contain strategy-revealing intermediate states that meaningfully constrain the continuation policy, resulting in weaker transfer when training Qwen.

A second practical implication is compute efficiency. Since PrefixRL amortizes training over a fixed pool of

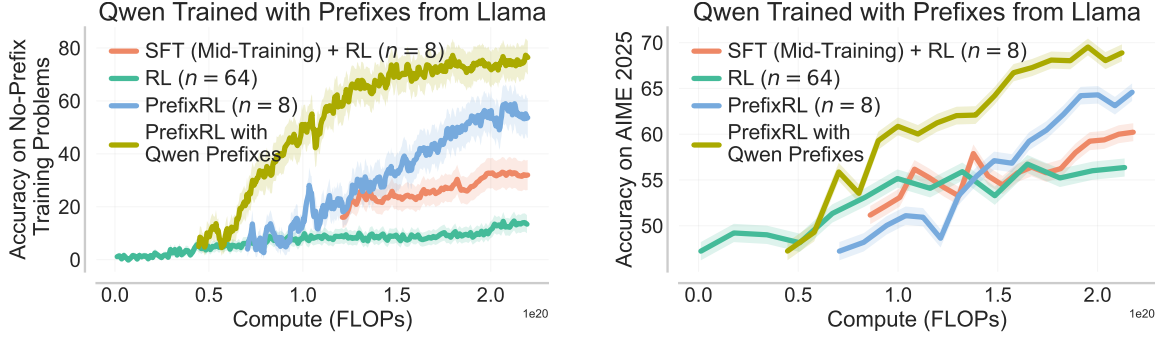


Figure 15 Training Qwen model with prefixes from Llama. Opposite of our experiment in Figure 11, here we train Qwen3-4b-instruct base LLM using off-policy traces sourced by rejection sampling Llama-3.1-8b-instruct model. We find that while off-policy prefixes from Llama are not as effective at improving Qwen as the other way around. Moreover, we also end up spending more compute upfront on rejection sampling the traces with the less capable Llama base model. We show accuracy on no-prefix training problems (*left*) and AIME 2025 (*right*).

off-policy traces, the total compute depends not only on the on-policy RL phase but also on the *upfront* cost of harvesting correct traces. Rejection sampling from the weaker base model increases this upfront cost, and Figure 15 shows that even after paying that cost, the resulting prefixes yield smaller downstream gains for Qwen. Together with Figure 14, these results suggest that cross-family prefix sourcing is most attractive when (a) the source model is strong enough to produce diverse correct traces at reasonable cost, and (b) the injected prefix states align with the target model sufficiently well to allow back-generalization to propagate to the no-prefix distribution.